

PYTHON EXERCISE

NAME: POOJASRI L

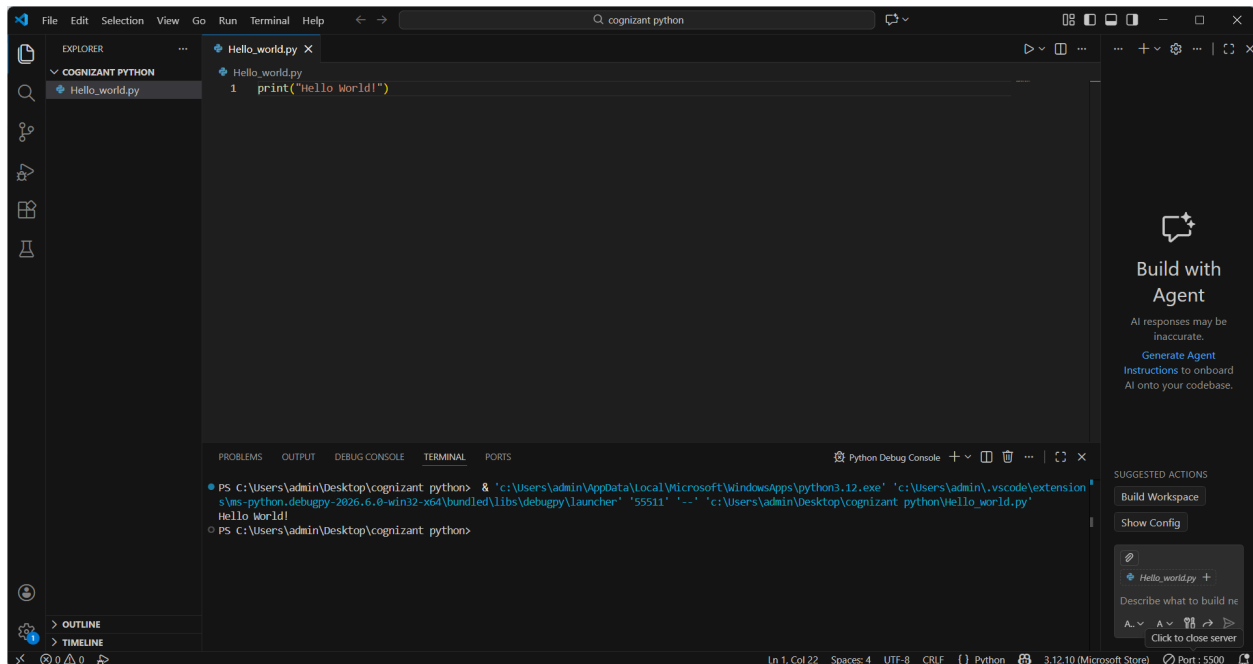
REGISTER NUMBER: 212223220076

BASIC CONCEPTS HANDS-ON QUESTIONS

Installing Python & Running Programs

1. Simple Hello World

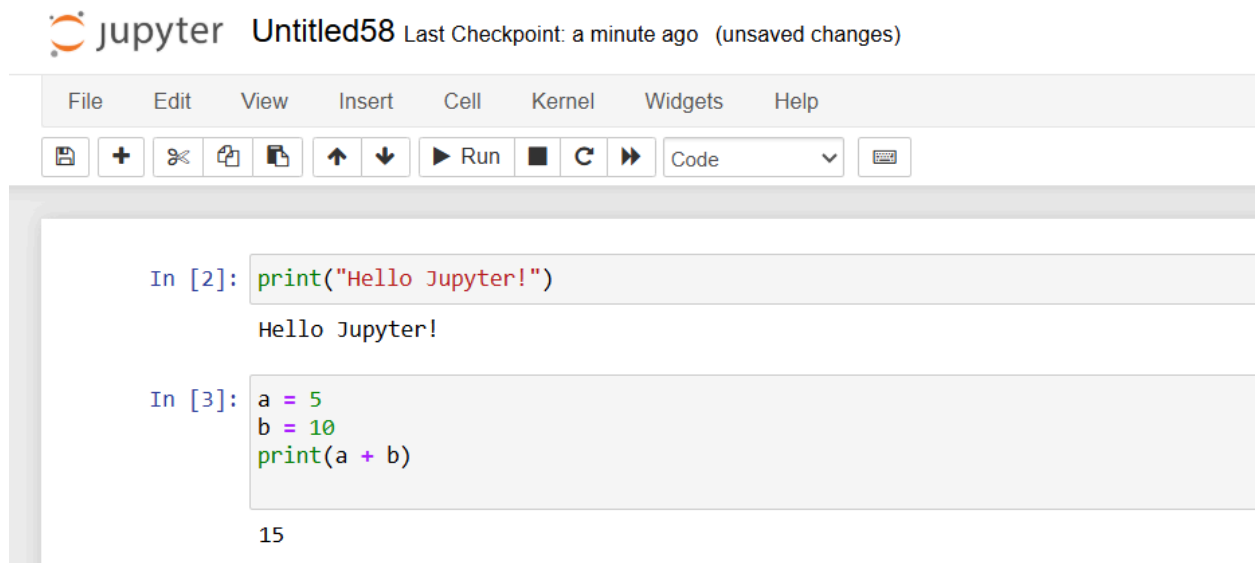
- **Objective:** Create first Python file.
- **Task:** Write and execute hello program.
- **Instructions:**
 - Create first.py with `print("Hello World!")`.
 - Run ***python first.py***.
 - Share terminal output.



Text Editors and IDEs

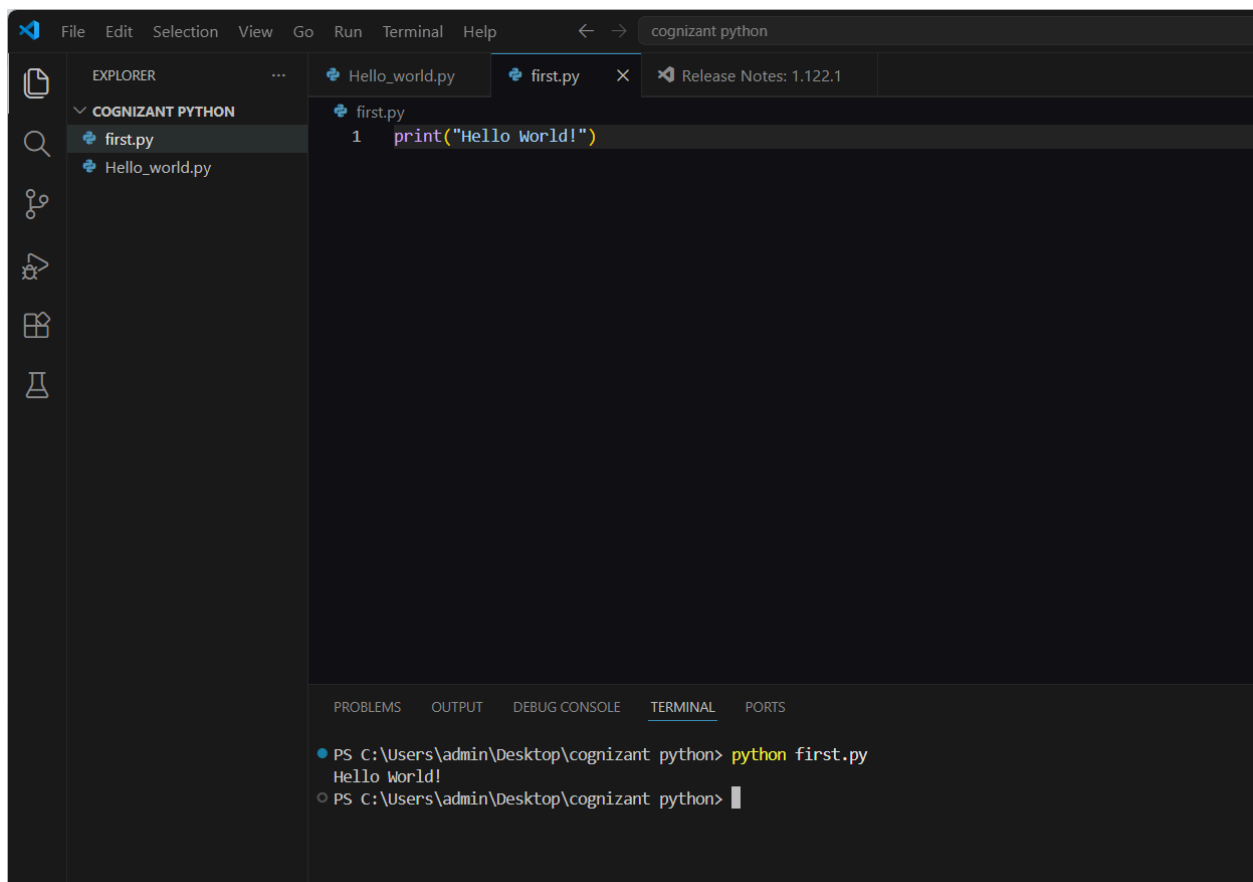
2. Jupyter Notebook

- **Objective:** Create Jupyter notebook.
- **Task:** Basic notebook operations.
- **Instructions:**
 - Run ***pip install notebook***, then ***jupyter notebook***.
 - Create new notebook, add code/math cells.



3. VS Code Setup

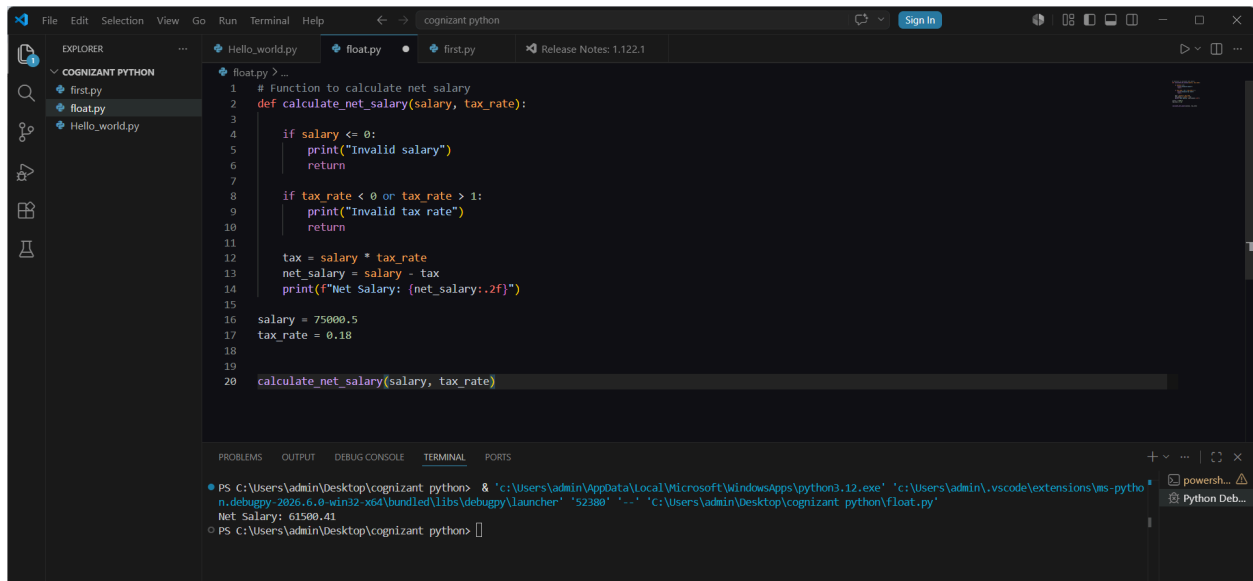
- **Objective:** Configure Python in VS Code.
- **Task:** Install Python extension.
- **Instructions:**
 - Install VS Code Python extension.
 - Create .py file, verify IntelliSense.
 - Run with Ctrl+F5.



Data Types

4. Float Precision

- **Objective:** Handle floating-point arithmetic with functions + validation + formatted output.
- **Task:** Calculate net salary after tax and print with 2 decimals.
- **Instructions:**
 - salary = 75000.5, tax_rate = 0.18.
 - Use a function
 - Validate salary & tax rate
 - Use variables
 - Format output to 2 decimal places



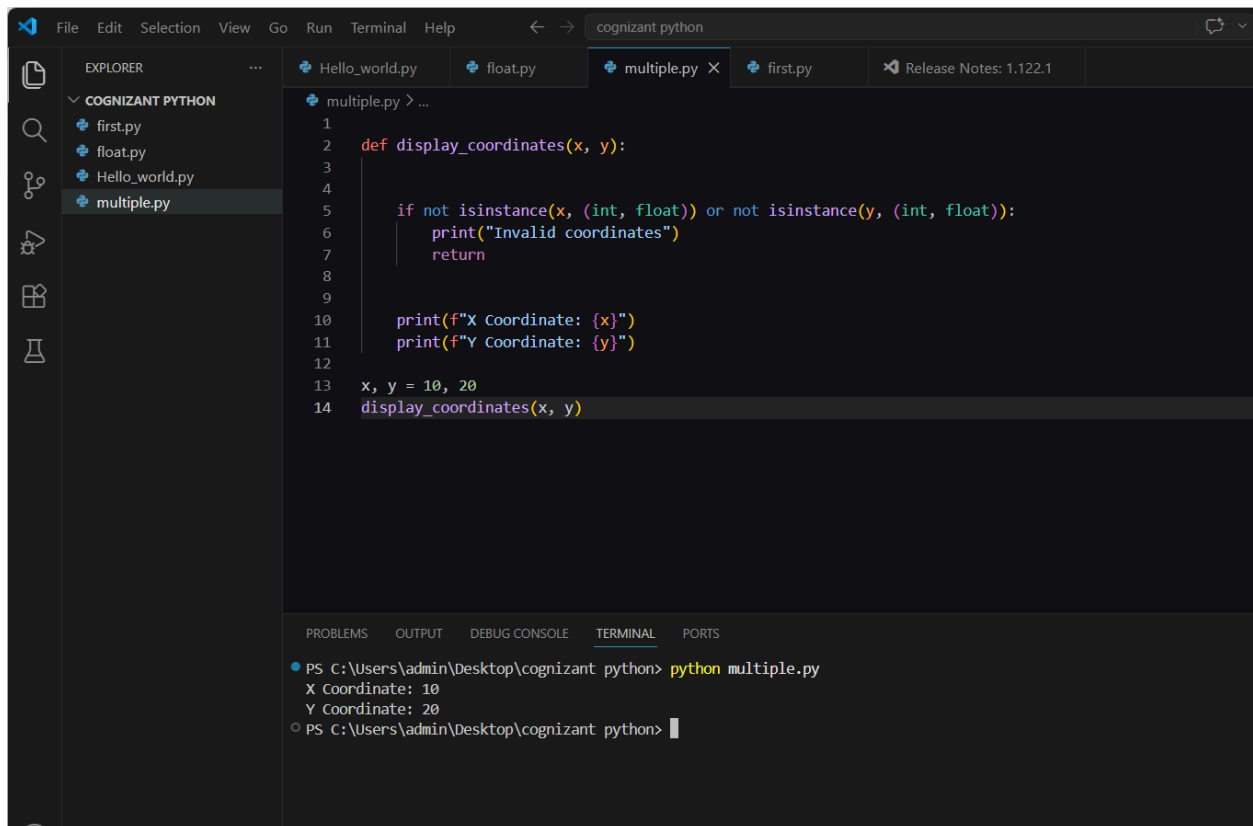
```
File Edit Selection View Go Run Terminal Help cognizant.python
EXPLORER
  Hello_world.py
  float.py
  first.py
  Release Notes: 1.122.1
COGNIZANT PYTHON
  first.py
  float.py
  Hello_world.py
float.py
1 # Function to calculate net salary
2 def calculate_net_salary(salary, tax_rate):
3
4     if salary <= 0:
5         print("Invalid salary")
6         return
7
8     if tax_rate < 0 or tax_rate > 1:
9         print("Invalid tax rate")
10        return
11
12    tax = salary * tax_rate
13    net_salary = salary - tax
14    print(f"Net Salary: {net_salary:.2f}")
15
16    salary = 75000.5
17    tax_rate = 0.18
18
19
20 calculate_net_salary(salary, tax_rate)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\admin\Desktop\cognizant_python> & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python-debugpy-2026.4.0-win32-x64\bundle\libs\debugpy\launcher' '52380' '-.' 'C:\Users\admin\Desktop\cognizant_python\float.py'
Net Salary: 61500.41
PS C:\Users\admin\Desktop\cognizant_python>
```

Variables

5. Multiple Assignment

- **Objective:** Use multiple assignment with basic validation + function + formatted output.
- **Task:** Unpack (x, y) coordinates and display them.
- **Instructions:**
 - Use a function
 - Validate coordinates
 - Use multiple assignment
 - Print coordinates



The screenshot shows the Visual Studio Code interface with a Python file named `multiple.py` open. The file contains a function `display_coordinates(x, y)` that validates the input types and prints the coordinates. The main code assigns `x = 10` and `y = 20`, then calls the function. The terminal at the bottom shows the command `python multiple.py` being executed, resulting in the output: `X Coordinate: 10` and `Y Coordinate: 20`.

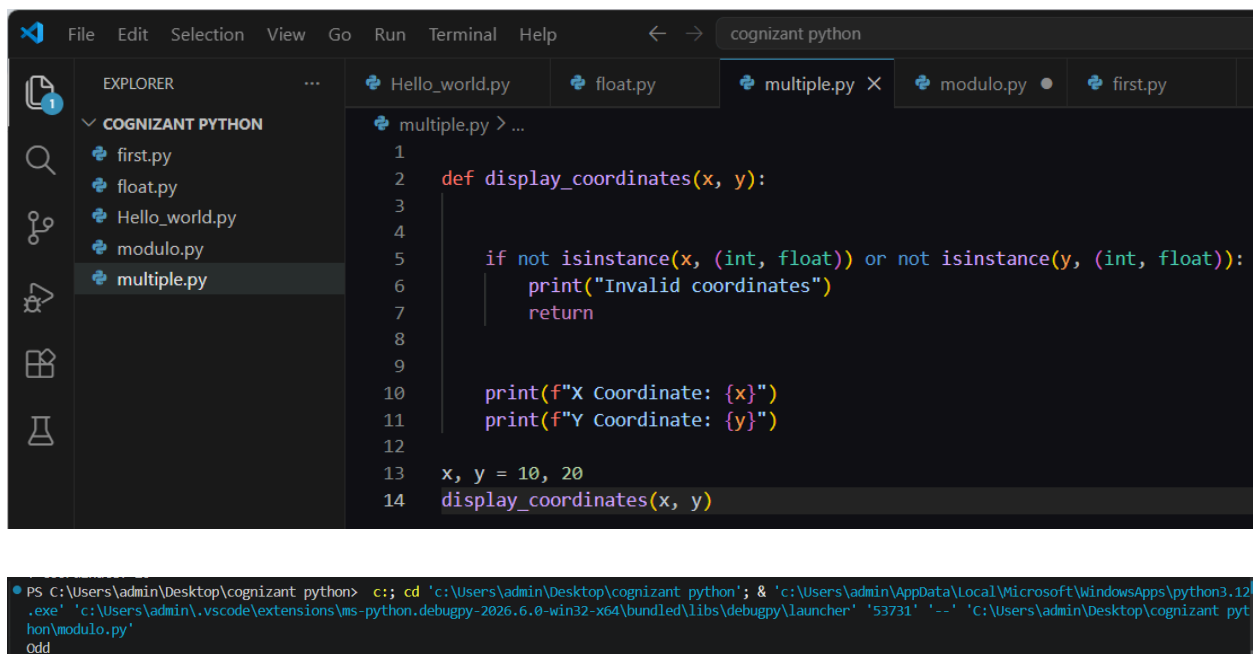
```
1
2 def display_coordinates(x, y):
3
4     if not isinstance(x, (int, float)) or not isinstance(y, (int, float)):
5         print("Invalid coordinates")
6         return
7
8
9
10    print(f"X Coordinate: {x}")
11    print(f"Y Coordinate: {y}")
12
13    x, y = 10, 20
14    display_coordinates(x, y)
```

```
PS C:\Users\admin\Desktop\cognizant python> python multiple.py
X Coordinate: 10
Y Coordinate: 20
PS C:\Users\admin\Desktop\cognizant python>
```

Mathematical Operations

6. Modulo Operator

- **Objective:** Use the modulo operator with function + validation + clean output.
- **Task:** Check if a number is even or odd using %.
- **Instructions:**
 - Use a function
 - Validate number
 - Compute remainder
 - Print “Even” or “Odd”
 - Use variable number = 17



The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left shows a project named 'COGNIZANT PYTHON' with files: first.py, float.py, Hello_world.py, modulo.py, and multiple.py. The main editor window displays the code for 'multiple.py'. The code defines a function 'display_coordinates(x, y)' that checks if both x and y are instances of int or float. If not, it prints 'Invalid coordinates' and returns. Otherwise, it prints the X and Y coordinates. At the bottom, there is a terminal window showing the command prompt output: 'PS C:\Users\admin\Desktop\cognizant python> c:; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '53731' '--' 'C:\Users\admin\Desktop\cognizant python\modulo.py' Odd'.

```
1
2 def display_coordinates(x, y):
3
4     if not isinstance(x, (int, float)) or not isinstance(y, (int, float)):
5         print("Invalid coordinates")
6         return
7
8
9
10    print(f"X Coordinate: {x}")
11    print(f"Y Coordinate: {y}")
12
13    x, y = 10, 20
14    display_coordinates(x, y)
```

```
PS C:\Users\admin\Desktop\cognizant python> c:; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '53731' '--' 'C:\Users\admin\Desktop\cognizant python\modulo.py'
Odd
```

7. Floor Division

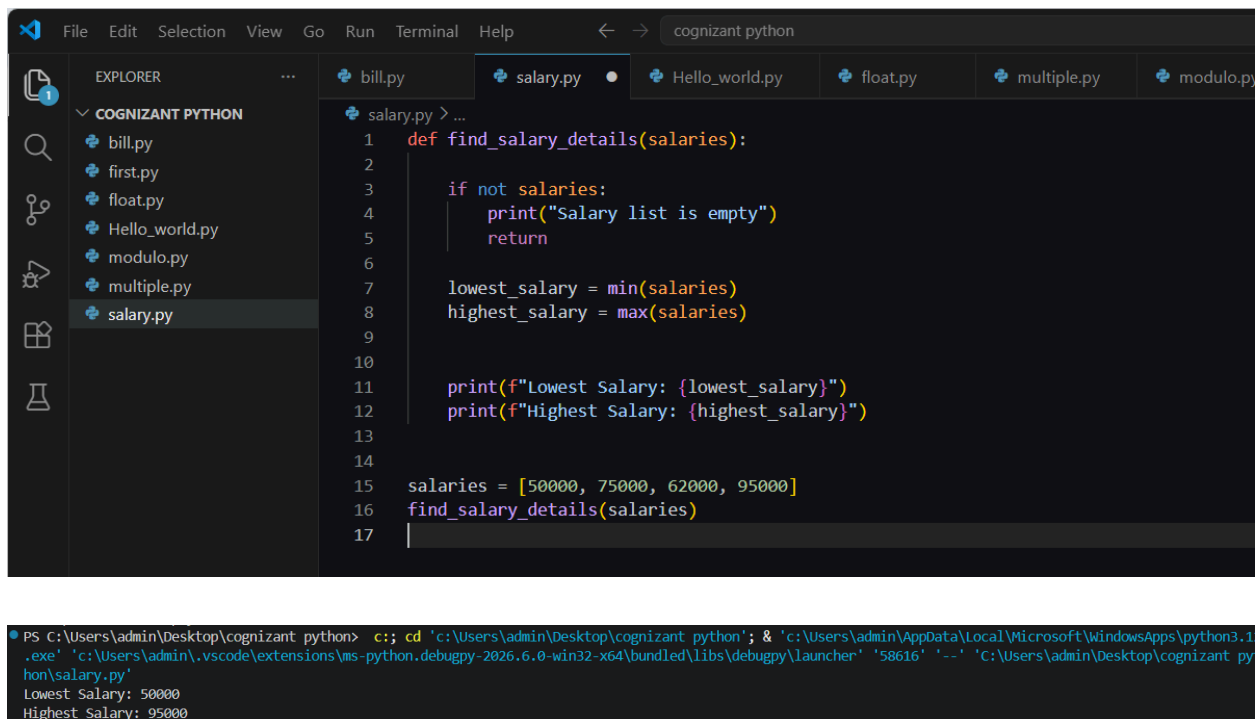
- **Objective:** Perform integer division with function + validation + formatted output.
- **Task:** Split a total bill equally among people.
- **Instructions:**
 - Use a function
 - Validate bill & number of people
 - Use floor division //
 - Print individual share
 - Variables: total_bill = 1250, people = 4

```
1
2 def split_bill(total_bill, people):
3
4     if total_bill <= 0 or people <= 0:
5         print("Invalid input")
6         return
7     share = total_bill // people
8
9     print(f"Each person should pay: {share}")
10
11 total_bill = 1250
12 people = 4
13 split_bill(total_bill, people)
```

```
PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12
.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '58739' '--' 'C:\Users\admin\Desktop\cognizant py
hon\bill.py'
Each person should pay: 312
```

8. Min/Max Functions

- **Objective:** Use built-in min() and max() with validation + function + structured output.
- **Task:** Find the highest and lowest salary in a list.
- **Instructions:**
 - Use a function
 - Validate the salary list
 - Compute min & max
 - Print the results
 - Use list: [50000, 75000, 62000, 95000]



The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left shows a project named 'COGNIZANT PYTHON' with several Python files. The 'salary.py' file is selected and open in the editor. The code in 'salary.py' defines a function 'find_salary_details(salaries)' that checks if the 'salaries' list is empty. If empty, it prints 'Salary list is empty' and returns. Otherwise, it calculates the minimum and maximum salary and prints them. The terminal at the bottom shows the command to run the script, which outputs the lowest and highest salaries from a predefined list.

```
File Edit Selection View Go Run Terminal Help
cognizant python

EXPLORER
COGNIZANT PYTHON
bill.py
first.py
float.py
Hello_world.py
modulo.py
multiple.py
salary.py

salary.py > ...
1 def find_salary_details(salaries):
2
3     if not salaries:
4         print("Salary list is empty")
5         return
6
7     lowest_salary = min(salaries)
8     highest_salary = max(salaries)
9
10
11     print(f"Lowest Salary: {lowest_salary}")
12     print(f"Highest Salary: {highest_salary}")
13
14
15 salaries = [50000, 75000, 62000, 95000]
16 find_salary_details(salaries)
17

PS C:\Users\admin\Desktop\cognizant python> c::; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.1
.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '58616' '--' 'C:\Users\admin\Desktop\cognizant py
hon\s salary.py'
Lowest Salary: 50000
Highest Salary: 95000
```

User Input

9. Basic Input

- **Objective:** Read and validate user input using a function + formatted output.
- **Task:** Get the user's name and greet them.
- **Instructions:**
 - Use a function
 - Validate empty input
 - Use input()
 - Print greeting message

The screenshot shows the Visual Studio Code interface. The Explorer panel on the left lists files in a folder named 'COGNIZANT PYTHON', including 'bill.py', 'first.py', 'float.py', 'Hello_world.py', 'modulo.py', 'multiple.py', 'salary.py', and 'user.py'. The 'user.py' file is open in the editor, showing the following code:

```
1
2 def greet_user():
3
4     name = input("Enter your name: ")
5
6     if name.strip() == "":
7         print("Name cannot be empty")
8         return
9
10    print(f"Hello, {name}!")
11
12 greet_user()
```

Below the editor, a terminal window shows the command prompt execution:

```
PS C:\Users\admin\Desktop\cognizant python> c::; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12\python.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52537' '--' 'c:\Users\admin\Desktop\cognizant python\user.py'
Enter your name: poojasri
Hello, poojasri!
```

10. Numeric Input

- **Objective:** Convert user input to a number with validation + function + formatted output.
- **Task:** Ask user for age, validate, convert to integer, and print next year's age.
- **Instructions:**
 - Use a function
 - Validate numeric input
 - Convert using `int()`
 - Print "Next year you'll be <age+1>"

The screenshot shows the Visual Studio Code interface with a project named 'cognizant python'. The Explorer sidebar on the left lists several Python files: age.py, bill.py, first.py, float.py, Hello_world.py, modulo.py, multiple.py, salary.py, and user.py. The main editor window displays the code for 'age.py', which defines a function 'next_year_age()' that takes an age as input, validates it, and prints the next year's age. The code includes a try-except block to handle 'ValueError' exceptions.

```
1
2 def next_year_age():
3
4     try:
5
6         age = int(input("Enter your age: "))
7
8         if age < 0:
9             print("Invalid age")
10            return
11
12            print(f"Next year you'll be {age + 1}")
13
14        except ValueError:
15            print("Please enter a valid number")
16
17    next_year_age()
18
```

Below the editor, a terminal window shows the command prompt output for running the script:

```
PS C:\Users\admin\Desktop\cognizant python> c::; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '57494' '--' 'C:\Users\admin\Desktop\cognizant python\age.py'
Enter your age: 20
Next year you'll be 21
```

11. Float Input

- **Objective:** Handle decimal input with validation + function + formatted output.
- **Task:** Convert weight from kilograms to pounds.
- **Instructions:**
 - Use a function
 - Validate decimal input
 - Convert kg → lbs using “lbs = kg * 2.20462”
 - Print the weight in pounds.

The screenshot shows the Visual Studio Code editor with a project named 'cognizant python'. The Explorer sidebar on the left lists several Python files: age.py, bill.py, first.py, float.py, Hello_world.py, modulo.py, multiple.py, salary.py, user.py, and weight.py. The 'weight.py' file is selected and open in the main editor. The code in 'weight.py' is as follows:

```
1
2 def convert_weight():
3
4     try:
5         kg = float(input("Enter weight in kilograms: "))
6
7         if kg <= 0:
8             print("Invalid weight")
9             return
10
11         lbs = kg * 2.20462
12         print(f"Weight in pounds: {lbs:.2f}")
13
14     except ValueError:
15         print("Please enter a valid decimal number")
16
17 convert_weight()
```

Below the editor, a terminal window shows the command prompt execution of the script:

```
PS C:\Users\admin\Desktop\cognizant python> c;; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.1
.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52932' '--' 'c:\Users\admin\Desktop\cognizant py
hon\weight.py'
Enter weight in kilograms: 52
Weight in pounds: 114.64
```

Control Flow

12. Simple If

- **Objective:** Basic conditional with function, validation, and user interaction.
- **Task:** Check if a student passes based on marks.
- **Instructions:**
 - Use a function
 - Validate marks
 - Use a variable (marks = 75)
 - Print Pass/Fail

```
File Edit Selection View Go Run Terminal Help
EXPLORER
COGNIZANT PYTHON
  age.py
  bill.py
  first.py
  float.py
  Hello_world.py
  marks.py
  modulo.py
  multiple.py
  salary.py
  user.py
  weight.py
marks.py
1 def check_result(marks):
2
3     if marks < 0 or marks > 100:
4         print("Invalid marks")
5         return
6
7     if marks >= 40:
8         print("Pass")
9     else:
10        print("Fail")
11
12
13 marks = 75
14
15 check_result(marks)

PS C:\Users\admin\Desktop\cognizant python> c::; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '57378' '--' 'c:\Users\admin\Desktop\cognizant python\marks.py'
Pass
```

13. If-Else

- **Objective:** Two-way decision using functions + validation + formatted output.
- **Task:** Check if a number is even or odd.
- **Instructions:**
 - Use a function
 - Validate input
 - Use variable num = 8
 - Print “Even” or “Odd”

```
1
2 def check_even_odd(num):
3
4     if not isinstance(num, int):
5         print("Invalid input")
6         return
7
8     if num % 2 == 0:
9         print("Even")
10    else:
11        print("Odd")
12
13
14 num = 8
15 check_even_odd(num)
16
```

```
PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12'
.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '64577' '--' 'C:\Users\admin\Desktop\cognizant pyt
hon\conditional.py'
Even
```

14. If-Elif-Else

- **Objective:** Multi-way decision using functions + validation + structured output.
- **Task:** Assign a grade based on the score.
- **Instructions:**
 - Use a function
 - Validate score
 - Use variable score = 88
- Print grade A/B/C
- Add slight enhancements over the basic version

The screenshot shows the Visual Studio Code editor with a file named `grade.py` open. The file contains the following Python code:

```
1
2 def assign_grade(score):
3
4     if score < 0 or score > 100:
5         print("Invalid score")
6         return
7
8     if score >= 90:
9         print("Grade A")
10    elif score >= 75:
11        print("Grade B")
12    else:
13        print("Grade C")
14
15    score = 88
16    assign_grade(score)
17
```

The Explorer sidebar on the left shows a folder named **COGNIZANT PYTHON** containing several files: `age.py`, `bill.py`, `conditional.py`, `first.py`, `float.py`, `grade.py` (selected), `Hello_world.py`, `marks.py`, `modulo.py`, `multiple.py`, `salary.py`, `user.py`, and `weight.py`.

Below the editor, a terminal window shows the command prompt output:

```
PS C:\Users\admin\Desktop\cognizant python> c:; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12\python.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '54271' '--' 'C:\Users\admin\Desktop\cognizant python\grade.py'
Grade B
```

15. Nested If

- **Objective:** Use nested conditions with a function + basic validation + formatted output.
- **Task:** Validate username and password using nested if statements.
- **Instructions:**
 - Use a function
 - Validate blank inputs
 - Nested if for user and password
 - Variables:
 - `user = "admin"`
 - `pwd = "pass123"`

```
File Edit Selection View Go Run Terminal Help
cognizant python

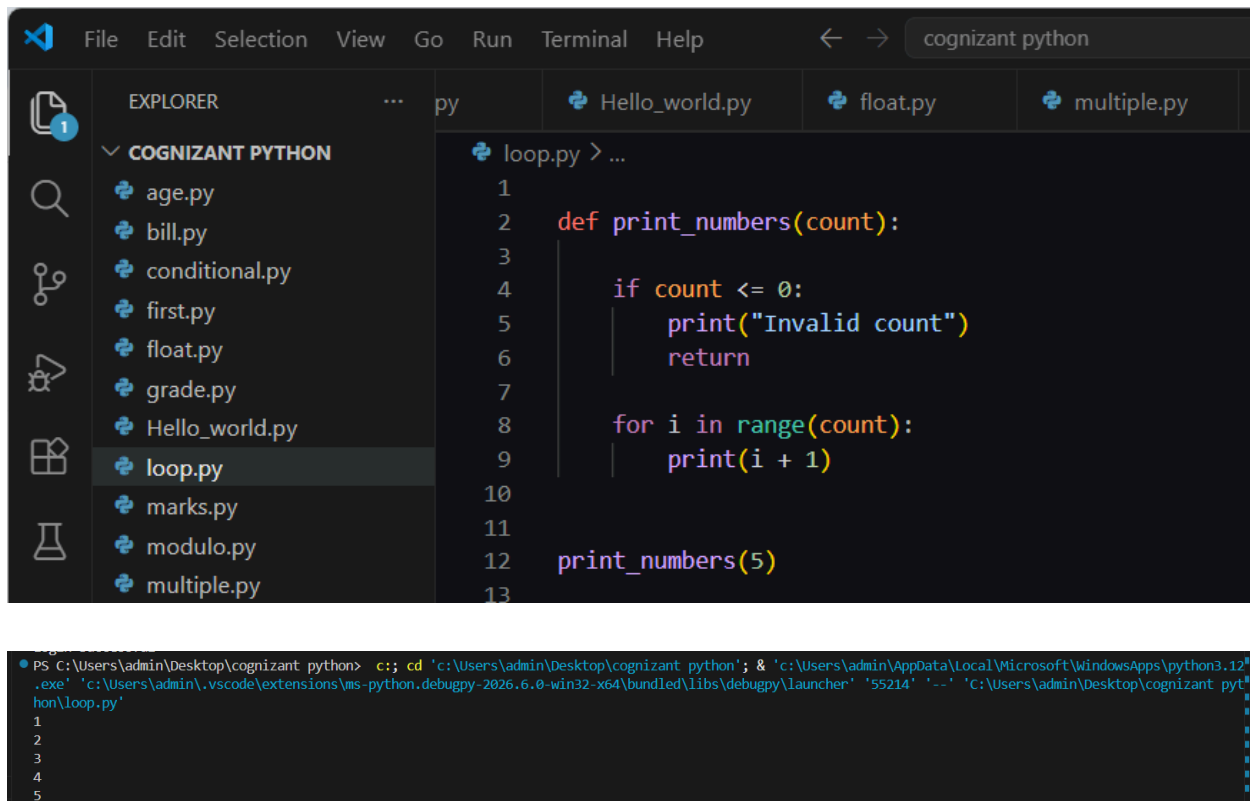
EXPLORER
COGNIZANT PYTHON
  age.py
  bill.py
  conditional.py
  first.py
  float.py
  grade.py
  Hello_world.py
  marks.py
  modulo.py
  multiple.py
  nested.py
  salary.py
  user.py
  weight.py

nested.py > ...
1 def login(user, pwd):
2
3     if user == "" or pwd == "":
4         print("Username or Password cannot be empty")
5         return
6
7     if user == "admin":
8         if pwd == "pass123":
9             print("Login Successful")
10        else:
11            print("Incorrect Password")
12    else:
13        print("Invalid Username")
14
15
16 user = "admin"
17 pwd = "pass123"
18 login(user, pwd)
19

PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12
.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '64198' '--' 'C:\Users\admin\Desktop\cognizant pyt
hon\nested.py'
Login Successful
```

16. For Loop Basics

- **Objective:** Iterate a fixed number of times with a function + validation + formatted output.
- **Task:** Print numbers from 1 to 5 using range().
- **Instructions:**
 - Use a function
 - Validate loop count
 - Use for i in range(5)
 - Print numbers 1–5



```
File Edit Selection View Go Run Terminal Help
EXPLORER
COGNIZANT PYTHON
  age.py
  bill.py
  conditional.py
  first.py
  float.py
  grade.py
  Hello_world.py
  loop.py
  marks.py
  modulo.py
  multiple.py
  loop.py > ...
1
2 def print_numbers(count):
3
4     if count <= 0:
5         print("Invalid count")
6         return
7
8     for i in range(count):
9         print(i + 1)
10
11
12 print_numbers(5)
13

PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12'
.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '55214' '--' 'c:\Users\admin\Desktop\cognizant pyth
hon\loop.py'
```

17. While Loop

- **Objective:** Use a condition-based loop with validation + function + formatted output.
- **Task:** Countdown from 5 to 1 using a while loop.
- **Instructions:**
 - Use a function
 - Validate count value
 - Use while count > 0:
 - Print count each step
 - Decrease using count -= 1

The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left lists files under the 'COGNIZANT PYTHON' workspace, including 'age.py', 'bill.py', 'conditional.py', 'first.py', 'float.py', 'grade.py', 'Hello_world.py', 'loop.py', and 'marks.py'. The main editor window displays the code for 'while_loop.py':

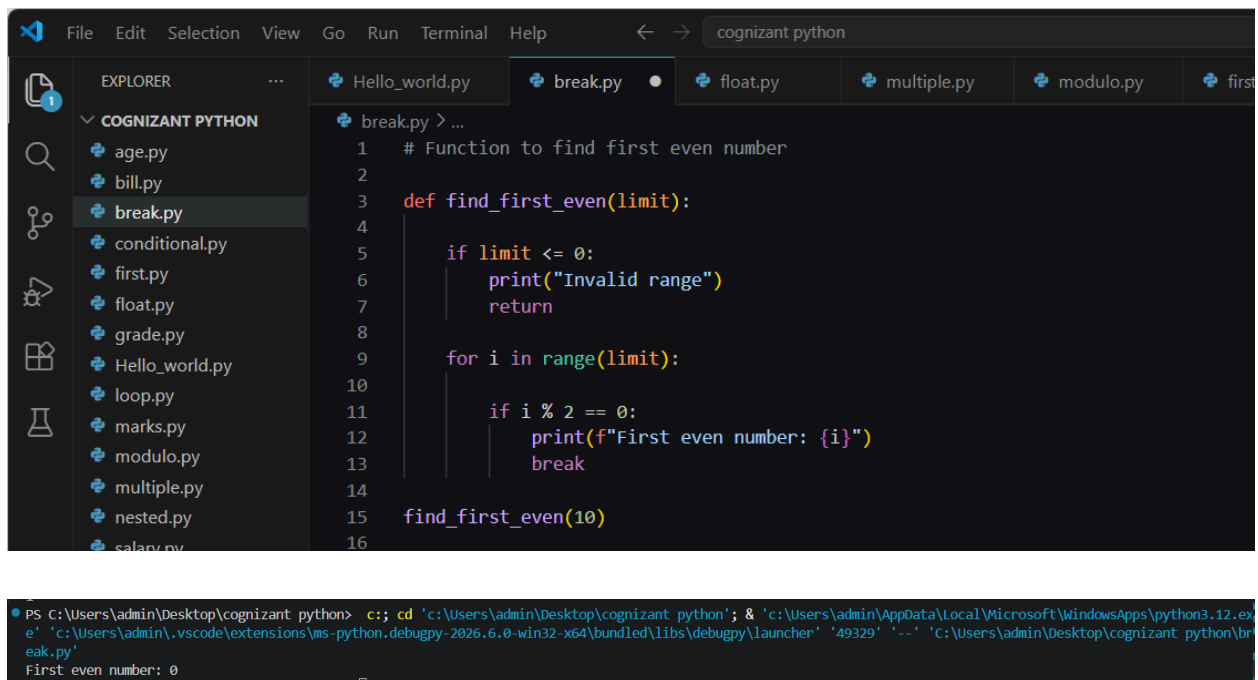
```
1 def countdown(count):
2     if count <= 0:
3         print("Invalid count")
4         return
5     while count > 0:
6         print(count)
7         count -= 1
8     countdown(5)
```

Below the editor, a terminal window shows the command prompt output:

```
PS C:\Users\admin\Desktop\cognizant python> c::; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '56019' '--' 'c:\Users\admin\Desktop\cognizant python\while_loop.py'
```

18. Break Statement

- **Objective:** Use a break statement to exit a loop early with function + validation + clean output.
- **Task:** Find and print the first even number in a given range.
- **Instructions:**
 - Use a function
 - Validate the range size
 - Loop with for i in range(...)
 - Use modulo % to check even
 - Break once the first even is found



```
File Edit Selection View Go Run Terminal Help
cognizant python

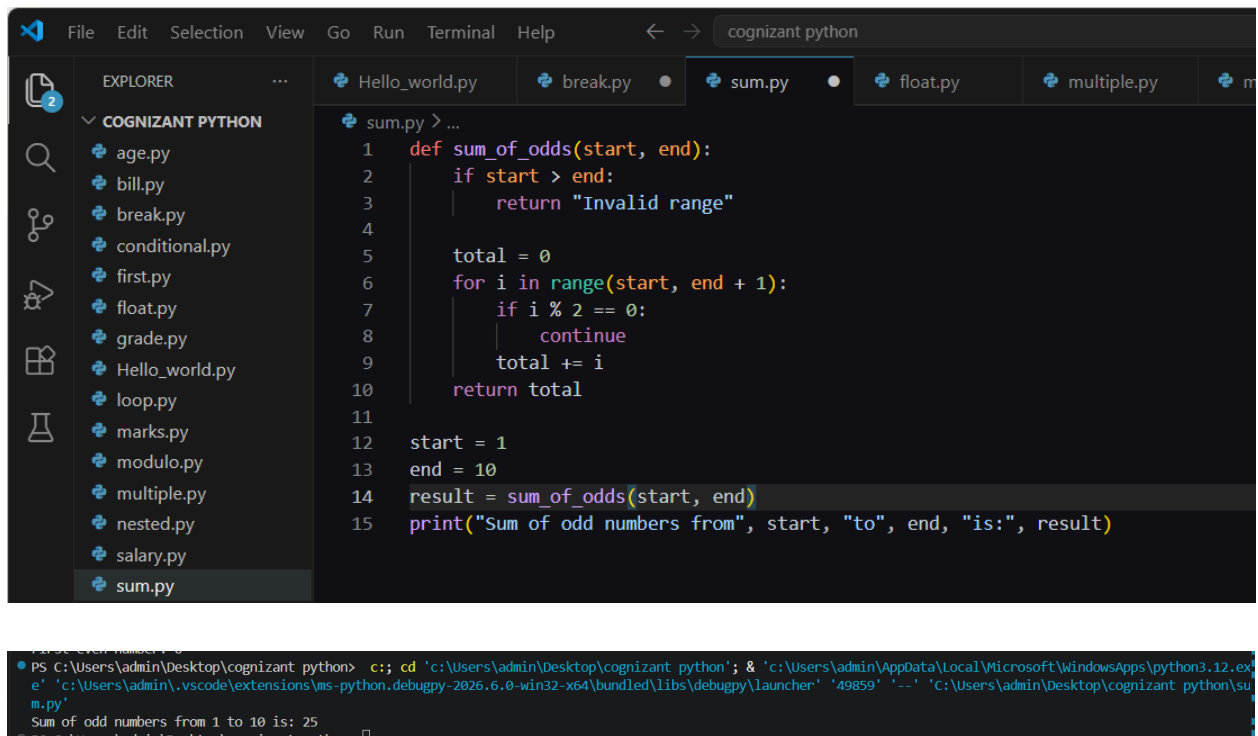
EXPLORER
COGNIZANT PYTHON
  age.py
  bill.py
  break.py
  conditional.py
  first.py
  float.py
  grade.py
  Hello_world.py
  loop.py
  marks.py
  modulo.py
  multiple.py
  nested.py
  salary.py

break.py > ...
1 # Function to find first even number
2
3 def find_first_even(limit):
4
5     if limit <= 0:
6         print("Invalid range")
7         return
8
9     for i in range(limit):
10
11         if i % 2 == 0:
12             print(f"First even number: {i}")
13             break
14
15 find_first_even(10)
16

PS C:\Users\admin\Desktop\cognizant python> c::; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '49329' '--' 'c:\Users\admin\Desktop\cognizant python\break.py'
First even number: 0
```

19. Continue Statement

- **Objective:** Use continue to skip even numbers, with a function + validation + formatted output.
- **Task:** Sum only the odd numbers in a given range.
- **Instructions:**
 - Use a function
 - Validate the range
 - Loop with range(10)
 - Skip even numbers using continue
 - Accumulate only odd numbers



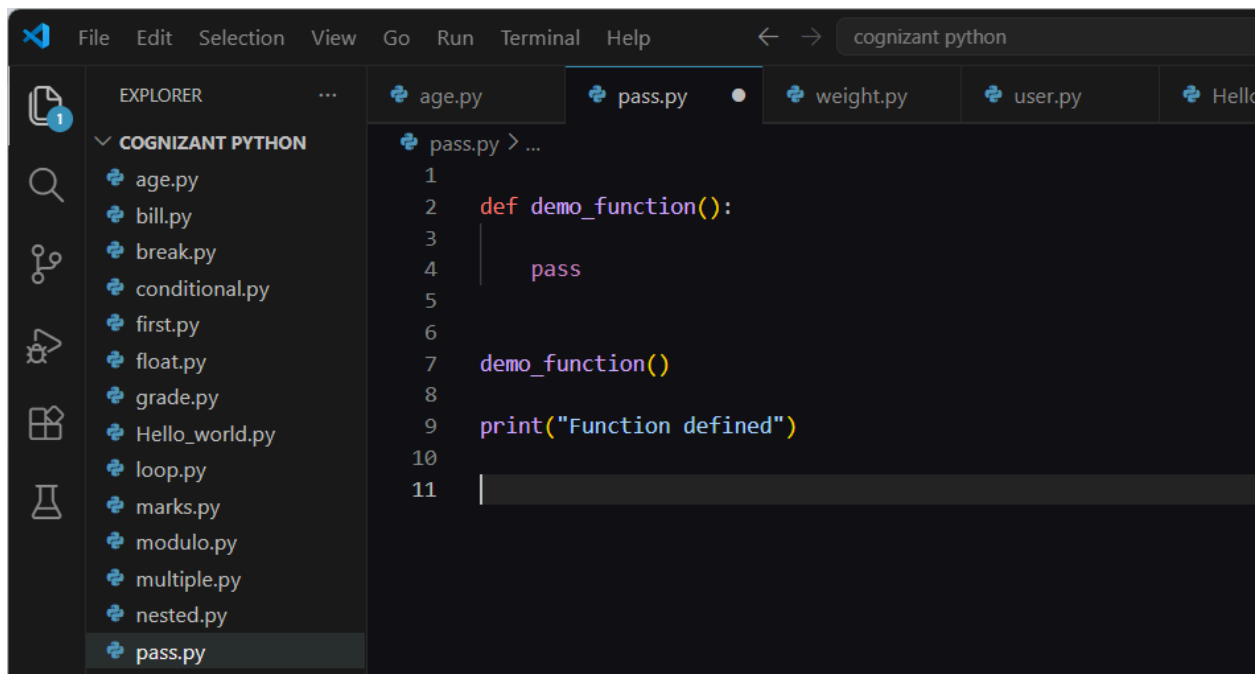
```
File Edit Selection View Go Run Terminal Help
cognizant python
EXPLORER
COGNIZANT PYTHON
age.py
bill.py
break.py
conditional.py
first.py
float.py
grade.py
Hello_world.py
loop.py
marks.py
modulo.py
multiple.py
nested.py
salary.py
sum.py

sum.py > ...
1 def sum_of_odds(start, end):
2     if start > end:
3         return "Invalid range"
4
5     total = 0
6     for i in range(start, end + 1):
7         if i % 2 == 0:
8             continue
9         total += i
10    return total
11
12 start = 1
13 end = 10
14 result = sum_of_odds(start, end)
15 print("Sum of odd numbers from", start, "to", end, "is:", result)

PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '49859' '...' 'C:\Users\admin\Desktop\cognizant python\sum.py'
Sum of odd numbers from 1 to 10 is: 25
```

20. Pass Statement

- **Objective:** Use pass as a placeholder inside a function, with simple structure and formatted output.
- **Task:** Define an empty function using pass and print confirmation.
- **Instructions:**
 - Use a function
 - Add placeholder logic with pass
 - Call the function
 - Print "Function defined"



```
PS C:\Users\admin\Desktop\cognizant python> c:: cd 'C:\Users\admin\Desktop\cognizant python'; & 'C:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'C:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '60404' '--' 'C:\Users\admin\Desktop\cognizant python\pass.py'
Function defined
```

Whitespace Importance

21. Consistent Indentation

- **Objective:** Demonstrate clean, consistent indentation using a nested if inside a function.
- **Task:** Print "Nested" when both conditions are True.
- **Instructions:**
 - Use a function
 - Apply exactly 4 spaces per indentation level
 - Use nested if-statements
 - Print "Nested"
 - Print confirmation message

The screenshot shows the Visual Studio Code interface with a file explorer on the left containing several Python files. The main editor displays the code for `consistent_info.py`, which defines a `check_nested(a, b)` function. The function uses nested if statements to check if both `a` and `b` are greater than 0. If both are true, it prints "Nested" and "Both conditions are True". The function is then called with `check_nested(5, 10)`. Below the editor, the terminal shows the command to run the script and its output: "Nested" and "Both conditions are True".

```
File Edit Selection View Go Run Terminal Help
cognizant python

EXPLORER
COGN...
  age.py
  bill.py
  break.py
  conditional.py
  consistent_info.py
  first.py
  float.py

consistent_info.py > ...
1 def check_nested(a, b):
2     if a > 0:
3         if b > 0:
4             print("Nested")
5             print("Both conditions are True")
6
7
8 check_nested(5, 10)
9

PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51572' '--' 'C:\Users\admin\Desktop\cognizant python\consistent_info.py'
Nested
Both conditions are True
```

Organizing Programs

22. Comment Usage

- **Objective:** Show proper documentation using meaningful comments + a function.
- **Task:** Add explanatory comments and print the total salary.
- **Instructions:**
 - Use clear comments
 - Use a function
 - Define base and bonus
 - Calculate total
 - Print result

The screenshot shows the Visual Studio Code interface with a file explorer on the left containing several Python files. The main editor displays the code for `common_usage.py`, which defines a `calculate_salary()` function. The function calculates the total salary by adding a base of 30000 and a bonus of 5000, then prints the result. The function is then called with `calculate_salary()`. Below the editor, the terminal shows the command to run the script and its output: "Total Salary: 35000".

```
File Edit Selection View Go Run Terminal Help
cognizant python

EXPLORER
COGNIZANT PYTHON
  age.py
  bill.py
  break.py
  common_usage.py
  conditional.py
  consistent_info.py
  first.py
  float.py
  grade.py

common_usage.py > ...
1
2
3 def calculate_salary():
4
5     base = 30000
6     bonus = 5000
7     total = base + bonus
8     print("Total Salary:", total)
9
10 calculate_salary()
11

PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51572' '--' 'C:\Users\admin\Desktop\cognizant python\common_usage.py'
Total Salary: 35000
```

```
PS C:\Users\admin\Desktop\cognizant python> c++; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '61854' '--' 'c:\Users\admin\Desktop\cognizant python\common_usage.py'
Total Salary: 35000
```

Modules

23. Import Standard Module

- **Objective:** Use the math module inside a function with validation + formatted output.
- **Task:** Calculate the area of a circle using math.pi and a given radius.
- **Instructions:**
 - Import math
 - Use a function
 - Validate radius

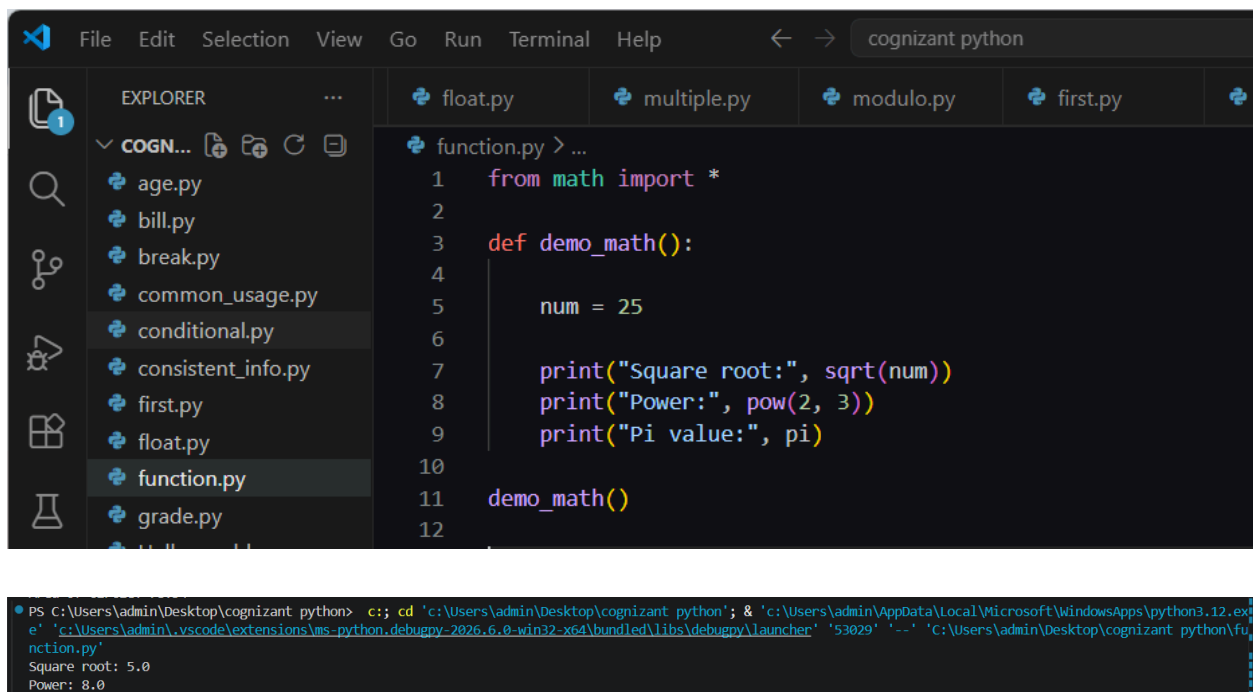
- Use formula: $\text{area} = \text{math.pi} \times \text{radius}^2$
- Print area with 2 decimals

```
module.py > ...
1  import math
2
3  def circle_area(radius):
4
5      if radius <= 0:
6          print("Invalid radius")
7          return
8
9      area = math.pi * radius ** 2
10     print(f"Area of circle: {area:.2f}")
11
12     circle_area(5)
13
14
```

```
PS C:\Users\admin\Desktop\cognizant python> c++; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '56625' '--' 'C:\Users\admin\Desktop\cognizant python\module.py'
Area of circle: 78.54
```

24. All Import

- **Objective:** Use from module import * along with simple utility usage inside a function.
- **Task:** Import everything from a standard module and use a few functions.
- **Instructions:**
 - Use from math import *
 - Use a function
 - Validate some input
 - Demonstrate usage (e.g., sqrt, pow, pi)
 - Print results



The screenshot shows the Visual Studio Code (VS Code) interface. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The Explorer panel on the left shows a project named 'COGNIZANT PYTHON' with several Python files: age.py, bill.py, break.py, common_usage.py, conditional.py, consistent_info.py, first.py, float.py, function.py (selected), and grade.py. The main editor area displays the code for 'function.py'. The code imports everything from the 'math' module and defines a function 'demo_math()' that calculates the square root of 25, the power of 2 to the 3rd, and prints the value of pi. The function is then called. Below the editor, the terminal shows the command to run the script and the output: 'Square root: 5.0' and 'Power: 8.0'.

```
File Edit Selection View Go Run Terminal Help cognizant python

EXPLORER
COGNIZANT PYTHON
  age.py
  bill.py
  break.py
  common_usage.py
  conditional.py
  consistent_info.py
  first.py
  float.py
  function.py
  grade.py

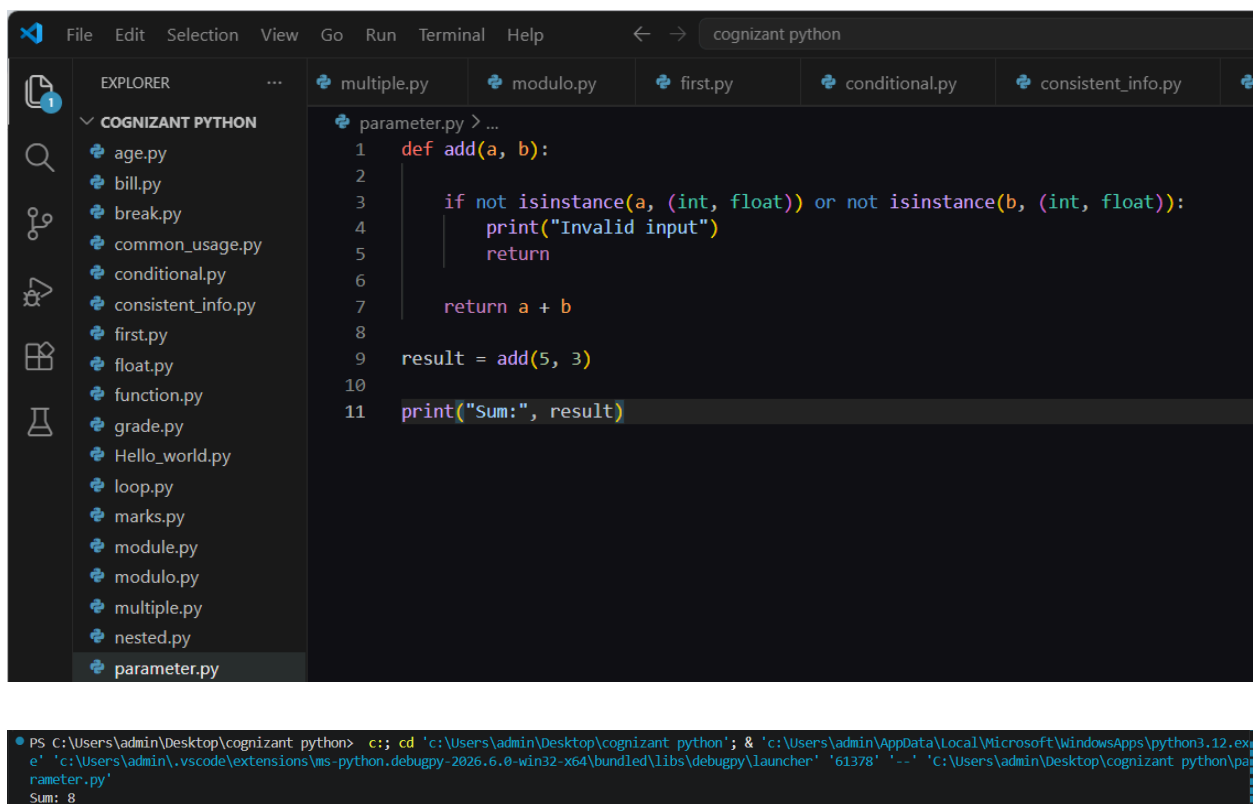
function.py > ...
1  from math import *
2
3  def demo_math():
4
5      num = 25
6
7      print("Square root:", sqrt(num))
8      print("Power:", pow(2, 3))
9      print("Pi value:", pi)
10
11  demo_math()
12

PS C:\Users\admin\Desktop\cognizant python> c::; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53029' '--' 'c:\Users\admin\Desktop\cognizant python\function.py'
Square root: 5.0
Power: 8.0
```

Functions

25. Parameters

- **Objective:** Pass arguments into a function with validation + structured output.
- **Task:** Create a function that adds two numbers and prints the result.
- **Instructions:**
 - Use a function with parameters
 - Validate inputs
 - Return the sum
 - Print the result of add(5, 3)



The screenshot shows the Visual Studio Code editor with a project named 'cognizant python'. The Explorer sidebar on the left lists several Python files, with 'parameter.py' selected. The main editor window displays the code in 'parameter.py':

```
1 def add(a, b):
2
3     if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
4         print("Invalid input")
5         return
6
7     return a + b
8
9 result = add(5, 3)
10
11 print("Sum:", result)
```

At the bottom, the integrated terminal shows the command used to run the script and its output:

```
PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '61378' '--' 'c:\Users\admin\Desktop\cognizant python\parameter.py'
Sum: 8
```


26. Multiple Parameters

- **Objective:** Handle multiple arguments in a function with validation + formatted output.
- **Task:** Calculate the area of a rectangle using length and width.
- **Instructions:**
 - Use a function that accepts two arguments
 - Validate the numbers
 - Calculate and return area
 - Print area(5, 3)



The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left displays a project named 'COGNIZANT PYTHON' with several Python files. The file 'area_rect.py' is selected and open in the editor. The code in 'area_rect.py' defines a function 'rectangle_area' that takes 'length' and 'width' as parameters. It includes a validation check: if either 'length' or 'width' is less than or equal to 0, it returns 'Invalid input'. Otherwise, it calculates the area as 'length * width' and returns it. At the bottom of the editor, a terminal window shows the command to run the script: 'python area_rect.py', which outputs '15'.

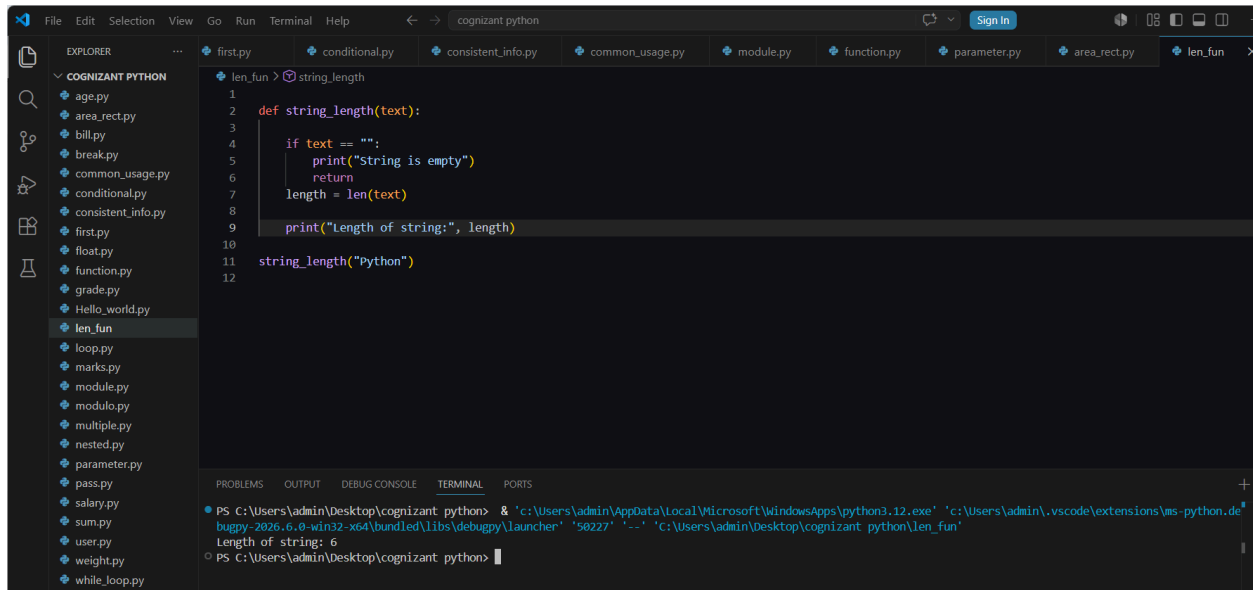
```
1 def rectangle_area(length, width):
2
3     if length <= 0 or width <= 0:
4         return "Invalid input"
5
6     area = length * width
7     return area
8
9
10 print(rectangle_area(5, 3))
```

```
PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '49680' '--' 'C:\Users\admin\Desktop\cognizant python\area_rect.py'
15
```

Built-in Functions

27. Len Function

- **Objective:** Use len() inside a function with basic validation + structured output.
- **Task:** Get the length of a given string.
- **Instructions:**
 - Use a function
 - Validate the string
 - Use len(text)
 - Print the length



The screenshot shows a VS Code editor window with the file explorer on the left displaying a project named 'COGNIZANT PYTHON'. The file 'len_fun.py' is selected. The editor shows the following code:

```
1 len_fun > string_length
2
3 def string_length(text):
4
5     if text == "":
6         print("String is empty")
7         return
8     length = len(text)
9
10    print("Length of string:", length)
11
12    string_length("Python")
```

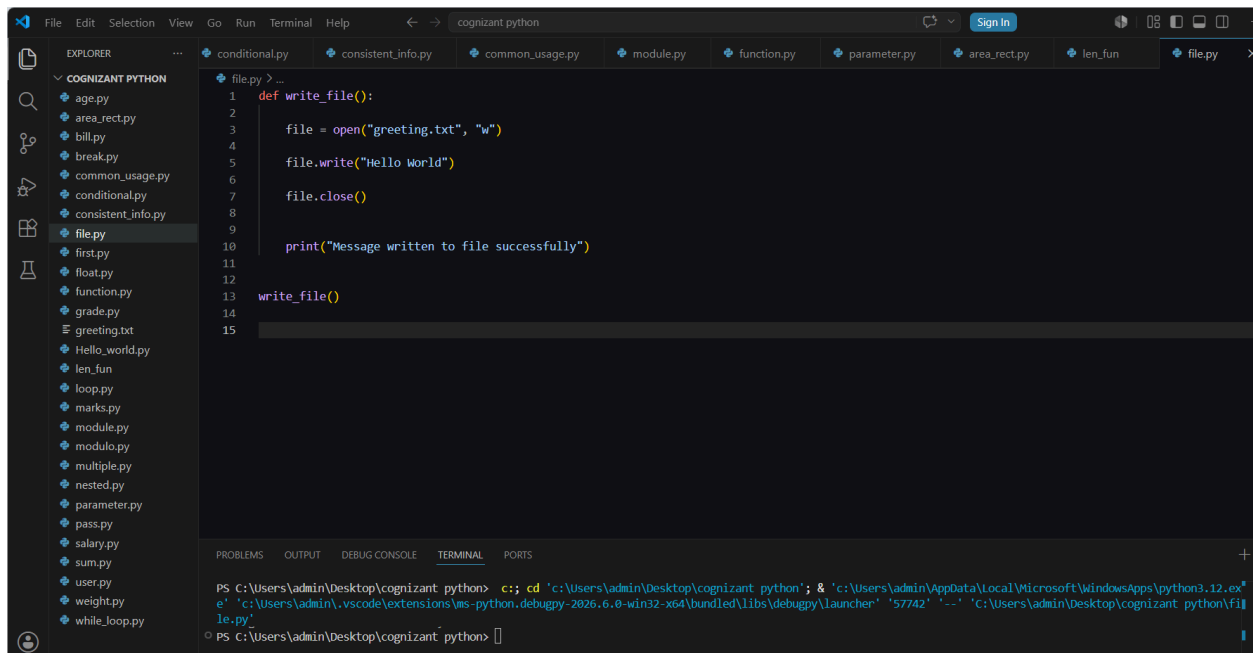
The terminal at the bottom shows the command to run the script and its output:

```
PS C:\Users\admin\Desktop\cognizant\python> & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.de
buggy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50227' '--' 'c:\Users\admin\Desktop\cognizant\python\len_fun'
Length of string: 6
PS C:\Users\admin\Desktop\cognizant\python>
```

File I/O

28. Write to File

- **Objective:** Create and write to a text file using a function + clean structure.
- **Task:** Save a greeting message into a text file.
- **Instructions:**
 - Use a function
 - Open a file in write mode
 - Write "Hello World" into the file
 - Print confirmation message



```
File Edit Selection View Go Run Terminal Help
cognizant python
EXPLORER
COGNIZANT PYTHON
  age.py
  area_rect.py
  bill.py
  break.py
  common_usage.py
  conditional.py
  consistent_info.py
  file.py
  first.py
  float.py
  function.py
  grade.py
  greeting.txt
  Hello_world.py
  len_fun
  loop.py
  marks.py
  module.py
  modulo.py
  multiple.py
  nested.py
  parameter.py
  pass.py
  salary.py
  sum.py
  user.py
  weight.py
  while_loop.py
file.py
1 def write_file():
2
3     file = open("greeting.txt", "w")
4
5     file.write("Hello World")
6
7     file.close()
8
9
10    print("Message written to file successfully")
11
12
13    write_file()
14
15
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\admin\Desktop\cognizant_python> c:; cd 'c:\Users\admin\Desktop\cognizant_python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '57742' '--' 'C:\Users\admin\Desktop\cognizant_python\file.py'
PS C:\Users\admin\Desktop\cognizant_python>
```

29. Read from File

- **Objective:** Read text from a file using a function with basic error handling.
- **Task:** Open a text file and display its contents.
- **Instructions:**
 - Use a function
 - Open file in read mode
 - Read full content using read()
 - Print file content
 - Handle missing file gracefully

```
1 def read_file():
2     try:
3         file = open("greeting.txt", "r")
4         content = file.read()
5         print("File Content:", content)
6         file.close()
7     except FileNotFoundError:
8         print("File not found")
9
10 read_file()
```

Terminal output:

```
PS C:\Users\admin\Desktop\cognizant_python> c:\cd 'c:\Users\admin\Desktop\cognizant_python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '50123' '--' 'c:\Users\admin\Desktop\cognizant_python\read_file.py'
File Content: Hello World
```

Error Handling

30. Basic Try-Except

- **Objective:** Catch simple runtime errors using try-except with a function and clear output.
- **Task:** Safely divide two numbers and handle division-by-zero errors.
- **Instructions:**
 - Use a function
 - Use try-except
 - Handle ZeroDivisionError
 - Print result or error message

```
File Edit Selection View Go Run Terminal Help
cognizant.python
EXPLORER
COGNIZANT PYTHON
  age.py
  area_rect.py
  bill.py
  break.py
  common_usage.py
  conditional.py
  consistent_info.py
  exception.py
  file.py
  first.py
  float.py
  function.py
  grade.py
  greeting.txt
  Hello_world.py
  len_fun
  loop.py
  marks.py
  module.py
  modulo.py
  multiple.py
  nested.py
  parameter.py
  pass.py
  read_file.py
  salary.py
  sum.py
  user.py
  weight.py
  while_loop.py
  common.py
exception.py
1 def divide_numbers(a, b):
2
3     try:
4
5         result = a / b
6
7         print("Result:", result)
8
9     except ZeroDivisionError:
10         print("Error: Cannot divide by zero")
11
12
13 divide_numbers(10, 2)
14 divide_numbers(10, 0)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\admin\Desktop\cognizant_python> c:\cd 'c:\Users\admin\Desktop\cognizant_python' & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '62521' '--' 'C:\Users\admin\Desktop\cognizant_python\exception.py'
Result: 5.0
Error: Cannot divide by zero
```

Lists

31. Create List

- **Objective:** Initialize a list with basic validation + a function + clean output.
- **Task:** Create a shopping cart list and display the items.
- **Instructions:**
 - Use a function
 - Validate list items
 - Initialize list with values [100, 250, 75]
 - Print the cart contents

```
1
2 def show_cart(cart):
3
4     if not cart:
5         print("Cart is empty")
6         return
7
8     print("Shopping Cart Items:", cart)
9
10 cart = [100, 250, 75]
11
12 show_cart(cart)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\admin\Desktop\cognizant_python> c:\cd 'c:\Users\admin\Desktop\cognizant_python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '60565' '--' 'C:\Users\admin\Desktop\cognizant_python\list.py'

Shopping Cart Items: [100, 250, 75]

PS C:\Users\admin\Desktop\cognizant_python>

32. Append to List

- **Objective:** Add a single item to a list using a function with validation.
- **Task:** Add a new expense to an existing expenses list.
- **Instructions:**
 - Use a function
 - Validate the expense amount

- Use .append()
- Print the updated expenses list

```
File Edit Selection View Go Run Terminal Help
cognizant python
EXPLORER
COGNIZANT PYTHON
  age.py
  append_list.py
  area_rect.py
  bill.py
  break.py
  common_usage.py
  conditional.py
  consistent_info.py
  exception.py
  file.py
  first.py
  float.py
  function.py
  grade.py
  greeting.txt
  Hello_world.py
  len_fun
  list.py
  loop.py
  marks.py
  module.py
  modulo.py
  multiple.py
  nested.py
  parameter.py
  pass.py
  read_file.py
  salary.py
  sum.py
  user.py
  OUTLINE
  ...
function.py
parameter.py
area_rect.py
len_fun
file.py
read_file.py
exception.py
list.py
append_list.py
1 def add_expense(expenses, amount):
2
3     if amount <= 0:
4         print("Invalid expense amount")
5         return
6
7     expenses.append(amount)
8
9     print("Updated Expenses List:", expenses)
10
11 expenses = [1000, 2000, 1500]
12
13 add_expense(expenses, 2500)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\admin\Desktop\cognizant_python> c:\cd 'c:\Users\admin\Desktop\cognizant_python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '54160' '--' 'C:\Users\admin\Desktop\cognizant_python\append_list.py'
Updated Expenses List: [1000, 2000, 1500, 2500]
```

Dictionaries

33. Update Dictionary

- **Objective:** Perform bulk dictionary updates using a function with basic validation.
- **Task:** Merge employee details from two dictionaries.
- **Instructions:**
 - Use a function
 - Validate dictionary inputs
 - Use .update()
 - Print the updated employee data

```
File Edit Selection View Go Run Terminal Help
cognizant.python
EXPLORER
COGNIZANT PYTHON
age.py
append_list.py
area_rect.py
bill.py
break.py
common_usage.py
conditional.py
consistent_info.py
dictionary.py
exception.py
file.py
first.py
float.py
function.py
grade.py
greeting.txt
Hello_world.py
len_fun
list.py
loop.py
marks.py
module.py
modulo.py
multiple.py
nested.py
parameter.py
pass.py
read_file.py
salary.py
sum.py
OUTLINE
parameter.py
area_rect.py
len_fun
file.py
read_file.py
exception.py
list.py
append_list.py
dictionary.py
update_employee_data
1 def update_employee_data(emp1, emp2):
2
3     if not isinstance(emp1, dict) or not isinstance(emp2, dict):
4         print("Invalid dictionary input")
5         return
6
7     emp1.update(emp2)
8
9     print("Updated Employee Data:", emp1)
10
11
12 employee1 = {"name": "Pooja", "salary": 50000}
13 employee2 = {"department": "IT", "city": "Chennai"}
14
15 update_employee_data(employee1, employee2)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\admin\Desktop\cognizant python> c:\cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53417' '--' 'c:\Users\admin\Desktop\cognizant python\dictionary.py'
Updated Employee Data: {'name': 'Pooja', 'salary': 50000, 'department': 'IT', 'city': 'Chennai'}
```

34. Nested Dictionary

- **Objective:** Work with nested dictionaries using a function and basic validation.
- **Task:** Store department-wise employee data and retrieve a specific employee's salary.
- **Instructions:**
 - Use a nested dictionary
 - Use a function to fetch data
 - Validate department and employee name
 - Print the employee salary

The screenshot shows a VS Code editor window with a file explorer on the left and a code editor in the center. The file explorer lists various Python files under the 'COGNIZANT PYTHON' folder, with 'nested_dict.py' selected. The code editor displays the following Python code:

```
1 def get_salary(data, department, employee):
2
3     if department not in data:
4         print("Department not found")
5         return
6
7     if employee not in data[department]:
8         print("Employee not found")
9         return
10
11     print("Employee Salary:", data[department][employee])
12
13 employees = {
14     "IT": {
15         "Pooja": 50000,
16         "Swetha": 60000
17     },
18     "HR": {
19         "Vasan": 45000
20     }
21 }
22
23 get_salary(employees, "IT", "Pooja")
```

The terminal at the bottom shows the command prompt output:

```
PS C:\Users\admin\Desktop\cognizant_python> c:\cd 'c:\Users\admin\Desktop\cognizant_python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '64638' '...' 'c:\Users\admin\Desktop\cognizant_python\nested_dict.py'
Employee Salary: 50000
```

Tuples and Sets

35. Create Tuple

- **Objective:** Use an immutable tuple with a function and basic validation.
- **Task:** Store fixed coordinates and display them.
- **Instructions:**
 - Use a tuple
 - Use a function
 - Validate coordinate values
 - Print coordinates in a readable format

```
File Edit Selection View Go Run Terminal Help cognizant python Sign In
EXPLORER
COGNIZANT PYTHON
conditional.py
consistent_info.py
dictionary.py
exception.py
file.py
first.py
float.py
function.py
grade.py
greeting.txt
Hello_world.py
len_fun
list.py
loop.py
marks.py
module.py
modulo.py
multiple.py
nested_dict.py
nested.py
parameter.py
pass.py
read_file.py
salary.py
sum.py
tuple.py
user.py
weight.py
while_loop.py
OUTLINE
TIMELINE

tuple.py > show_coordinates
1 def show_coordinates(coords):
2
3     if not isinstance(coords, tuple):
4         print("Invalid coordinates")
5         return
6
7     print(f"X Coordinate: {coords[0]}")
8     print(f"Y Coordinate: {coords[1]}")
9
10
11 coordinates = (10, 20)
12
13 show_coordinates(coordinates)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\admin\Desktop\cognizant_python> c::; cd 'C:\Users\admin\Desktop\cognizant_python'; & 'C:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'C:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '51690' '--' 'C:\Users\admin\Desktop\cognizant_python\tuple.py'
X Coordinate: 10
Y Coordinate: 20
PS C:\Users\admin\Desktop\cognizant_python>
```

36. Set Intersection

- **Objective:** Find common elements between two sets using a function and basic validation.
- **Task:** Identify common skills between two skill sets.
- **Instructions:**
 - Use sets
 - Use a function
 - Validate inputs
 - Find intersection using &
 - Print common skills

The screenshot shows a VS Code editor window with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'COGNIZANT PYTHON' with various files. The file 'set.py' is selected. The code in 'set.py' defines a function 'common_skills' that takes two sets as input and returns their intersection. The terminal shows the command to run the script and the output 'Common Skills: {'Python', 'SQL'}'.

```
1 def common_skills(set1, set2):
2
3     if not isinstance(set1, set) or not isinstance(set2, set):
4         print("Invalid input")
5         return
6
7     common = set1 & set2
8
9     print("Common Skills:", common)
10
11 skills1 = {"Python", "Java", "SQL"}
12 skills2 = {"Python", "C++", "SQL"}
13
14 common_skills(skills1, skills2)
```

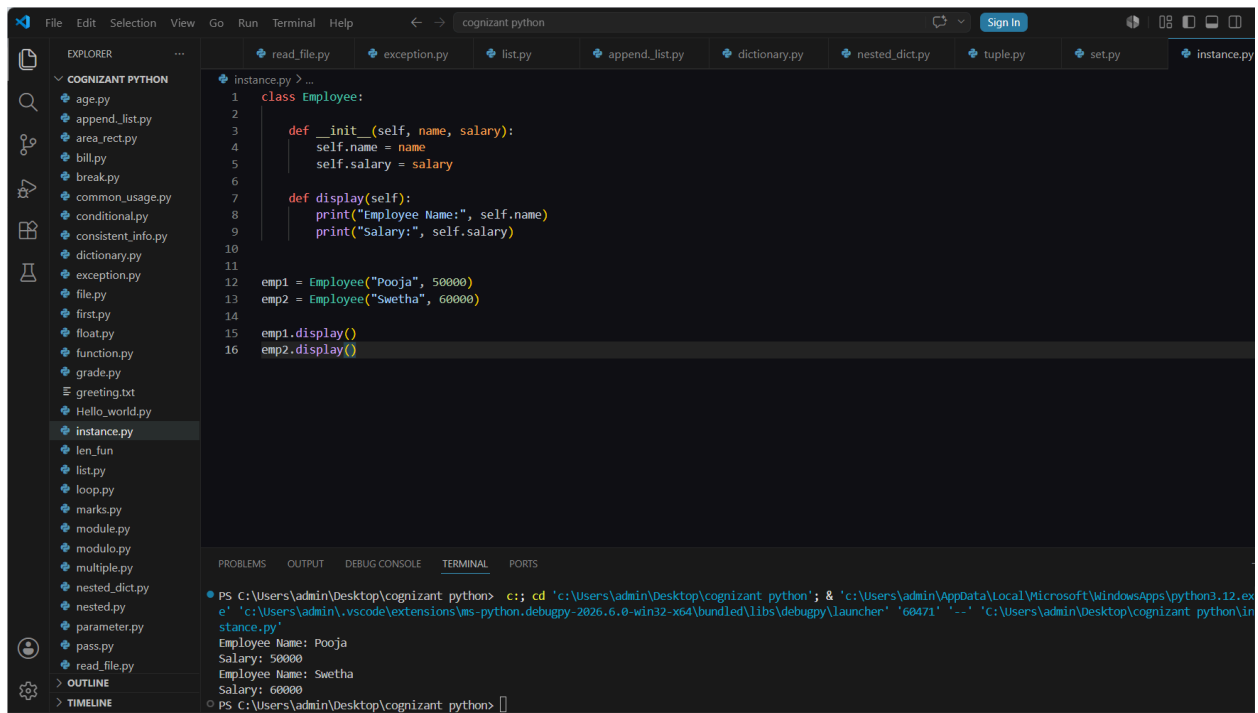
```
PS C:\Users\admin\Desktop\cognizant_python> cd 'c:\Users\admin\Desktop\cognizant_python' & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '64739' '-.' 'c:\Users\admin\Desktop\cognizant_python\set.py'
Common Skills: {'Python', 'SQL'}
PS C:\Users\admin\Desktop\cognizant_python>
```

OOP concepts

Classes

37. Multiple Instances

- **Objective:** Create multiple objects from a class and access their attributes.
- **Task:** Create an employee roster using multiple instances of a class.
- **Instructions:**
 - Define a class with `__init__`
 - Create multiple objects (instances)
 - Add a method to display employee info
 - Print employee names



```
File Edit Selection View Go Run Terminal Help cognizant python Sign In
EXPLORER
COGNIZANT PYTHON
age.py
append_list.py
area_rect.py
bill.py
break.py
common_usage.py
conditional.py
consistent_info.py
dictionary.py
exception.py
file.py
first.py
float.py
function.py
grade.py
greeting.txt
Hello_world.py
instance.py
len_fun
list.py
loop.py
marks.py
module.py
modulo.py
multiple.py
nested_dict.py
nested.py
parameter.py
pass.py
read_file.py
OUTLINE
TIMELINE

instance.py > ...
1 class Employee:
2
3     def __init__(self, name, salary):
4         self.name = name
5         self.salary = salary
6
7     def display(self):
8         print("Employee Name:", self.name)
9         print("Salary:", self.salary)
10
11
12 emp1 = Employee("Pooja", 50000)
13 emp2 = Employee("Swetha", 60000)
14
15 emp1.display()
16 emp2.display()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '60471' '-...' 'C:\Users\admin\Desktop\cognizant python\instance.py'
Employee Name: Pooja
Salary: 50000
Employee Name: Swetha
Salary: 60000
PS C:\Users\admin\Desktop\cognizant python> |
```

38. Method Chaining

- **Objective:** Demonstrate method chaining by returning self from methods.
- **Task:** Create an employee object, set salary, apply raise, and display final salary.
- **Instructions:**
 - Create a class
 - Methods should return self
 - Use method chaining
 - Add basic validation
 - Print final salary

```

1 class Employee:
2     def __init__(self, name):
3         self.name = name
4         self.salary = 0
5
6     def set_salary(self, salary):
7
8         if salary > 0:
9             self.salary = salary
10            return self
11
12    def apply_raise(self, amount):
13
14        if amount > 0:
15            self.salary += amount
16        return self
17
18    def display(self):
19        print("Employee Name:", self.name)
20        print("Final Salary:", self.salary)
21
22    emp = Employee("Poojasri")
23    emp.set_salary(50000).apply_raise(5000).display()

```

```

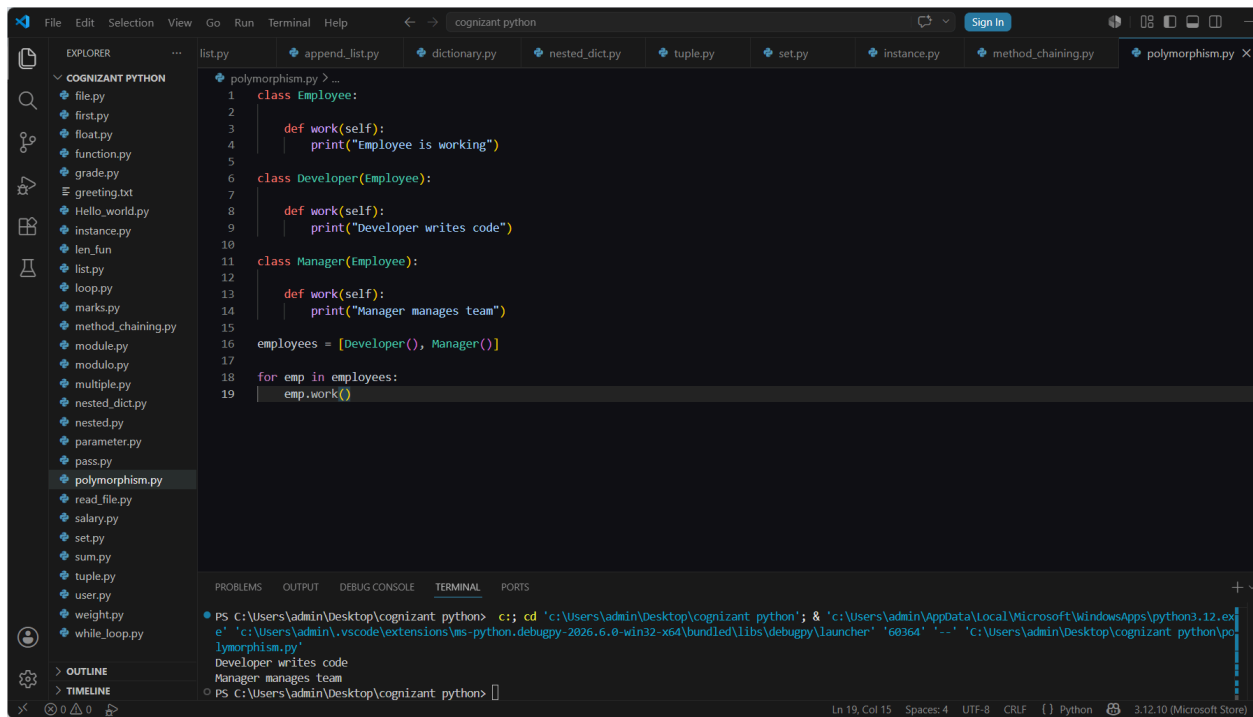
PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '49678' '--' 'c:\Users\admin\Desktop\cognizant python\method_chaining.py'
Employee Name: Poojasri
Final Salary: 55000
PS C:\Users\admin\Desktop\cognizant python>

```

Custom Classes & Inheritance

39. Polymorphism

- **Objective:** Demonstrate polymorphism using method overriding.
- **Task:** Show different behaviors of the same method (work()) for different employee types.
- **Instructions:**
 - Create a base class with a common method
 - Override the method in child classes
 - Store objects in a list
 - Call the same method for all objects
 - Observe different outputs



```
1 class Employee:
2     def work(self):
3         print("Employee is working")
4
5 class Developer(Employee):
6     def work(self):
7         print("Developer writes code")
8
9 class Manager(Employee):
10     def work(self):
11         print("Manager manages team")
12
13 employees = [Developer(), Manager()]
14
15 for emp in employees:
16     emp.work()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\admin\Desktop\cognizant_python> cd 'c:\Users\admin\Desktop\cognizant_python' & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '60364' '--' 'C:\Users\admin\Desktop\cognizant_python\polymorphism.py'

Developer writes code
Manager manages team

Ln 19, Col 15 Spaces: 4 UTF-8 CRLF Python 3.12.10 (Microsoft Store)

40. Class Methods

- **Objective:** Use a class method as a factory to create objects from a formatted string.
- **Task:** Create an Employee object from a string like "Shubh,75000".
- **Instructions:**
 - Use @classmethod
 - Parse string input
 - Convert salary to integer
 - Return a new object using cls
 - Print employee details

The screenshot shows a VS Code editor window with a file explorer on the left containing a folder named 'COGNIZANT PYTHON' with various Python files. The main editor displays the content of 'class_method.py'.

```

1 class Employee:
2
3     def __init__(self, name, salary):
4         self.name = name
5         self.salary = salary
6
7     @classmethod
8     def from_string(cls, emp_data):
9
10        name, salary = emp_data.split(",")
11
12        salary = int(salary)
13
14        return cls(name, salary)
15
16    def display(self):
17        print("Employee Name:", self.name)
18        print("Salary:", self.salary)
19
20
21 emp = Employee.from_string("Selvi,100000")
22 emp.display()

```

The terminal at the bottom shows the command to run the script and its output:

```

PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '65418' '--' 'c:\Users\admin\Desktop\cognizant python\class_method.py'
Employee Name: Selvi
Salary: 100000
PS C:\Users\admin\Desktop\cognizant python>

```

Real-time simulation hands-on questions

41. Employee Management System

- **Objective:** Use classes, dictionaries, lists, and JSON file I/O together.
- **Task:** Create an employee management system that can save and load employee data.
- **Instructions:**
 - Create Employee class with `__init__` and `__str__`
 - Store employees in a dictionary
 - Save data to `emps.json`
 - Load data back from file
 - Print all employees

```
1 import json
2 class Employee:
3     def __init__(self, emp_id, name, salary):
4         self.emp_id = emp_id
5         self.name = name
6         self.salary = salary
7     def __str__(self):
8         return f"ID: {self.emp_id}, Name: {self.name}, Salary: {self.salary}"
9
10 employees = {}
11 employees["E101"] = Employee("E101", "Pooja", 50000)
12 employees["E102"] = Employee("E102", "Swetha", 60000)
13 data = {}
14
15 for emp_id, emp in employees.items():
16     data[emp_id] = {
17         "name": emp.name,
18         "salary": emp.salary
19     }
20
21 file = open("emps.json", "w")
22 json.dump(data, file)
23 file.close()
24 print("Employee data saved successfully")
25
26 file = open("emps.json", "r")
27 loaded_data = json.load(file)
28 file.close()
29 print("\nEmployee Details:")
30 for emp_id, details in loaded_data.items():
31     print(f"ID: {emp_id}, Name: {details['name']}, Salary: {details['salary']}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
e' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '49722' '--' 'c:\Users\admin\Desktop\cognizant_python\emf
...
Employee Details:
ID: E101, Name: Pooja, Salary: 50000
ID: E102, Name: Swetha, Salary: 60000
PS C:\Users\admin\Desktop\cognizant_python>
```

42. Data Analysis Pipeline

- **Objective:** Use modules, functions, lists, file handling, and error handling together.
- **Task:** Process sales data from a file and calculate statistics.
- **Instructions:**
 - Import statistics module
 - Read numeric data from sales.txt
 - Handle file and data errors using try-except
 - Calculate mean and median
 - Print a statistics summary


```

File Edit Selection View Go Run Terminal Help
cognizant python
EXPLORER
COGNIZANT PYTHON
  age.py
  append_list.py
  area_rect.py
  bill.py
  break.py
  class_method.py
  common_usage.py
  conditional.py
  consistent_info.py
  data_analysis.py
  dictionary.py
  employee.py
  empsjson
  exception.py
  file.py
  first.py
  float.py
  function.py
  grade.py
  greeting.txt
  Hello_world.py
  instance.py
  len_fun
  list.py
  loop.py
  marks.py
  method_chaining.py
  module.py
  modulo.py
  multiple.py
  OUTLINE
  TIMELINE
  data_analysis.py
1 import statistics
2 def analyze_sales():
3     try:
4         file = open("sales.txt", "r")
5         data = file.readlines()
6         sales = [float(i.strip()) for i in data]
7         file.close()
8
9         mean_value = statistics.mean(sales)
10        median_value = statistics.median(sales)
11
12        print("Sales Data:", sales)
13        print("Mean:", mean_value)
14        print("Median:", median_value)
15
16    except FileNotFoundError:
17        print("File not found")
18
19    except ValueError:
20        print("Invalid data in file")
21
22    analyze_sales()

```

```

PS C:\Users\admin\Desktop\cognizant python> c:\cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50381' '--' 'c:\Users\admin\Desktop\cognizant python\data_analysis.py'
Sales Data: [1000.0, 2500.0, 3000.0, 1500.0, 2000.0]
Mean: 2000.0
Median: 2000.0

```

43. Configuration Manager

- **Objective:** Use classes, inheritance, dictionaries, and file I/O to manage application configuration.
- **Task:** Create a configurable system that loads database settings from a file.
- **Instructions:**
 - Create a base Config class
 - Create DatabaseConfig class inheriting from Config
 - Use configparser module
 - Load settings from db.ini
 - Validate required keys

```

1 import configparser
2 class Config:
3     pass
4 class DatabaseConfig(Config):
5     def load_config(self):
6
7         config = configparser.ConfigParser()
8         config.read("db.ini")
9
10        if "DATABASE" not in config:
11            print("DATABASE section not found")
12            return
13
14        required_keys = ["host", "user", "password"]
15        for key in required_keys:
16            if key not in config["DATABASE"]:
17                print(f"Missing key: {key}")
18                return
19
20        print("Database configuration")
21        print("Host:", config["DATABASE"]["host"])
22        print("User:", config["DATABASE"]["user"])
23        print("Password:", config["DATABASE"]["password"])
24
25        db = DatabaseConfig()
26        db.load_config()

```

```

PS C:\Users\admin\Desktop\cognizant python> c:\cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '54548' '--' 'c:\Users\admin\Desktop\cognizant python\manager.py'
Host: localhost
User: Poojasri
Password: pooja123
PS C:\Users\admin\Desktop\cognizant python>

```

44. CSV Data Processor

- **Objective:** Use file I/O, lists, dictionaries, and comprehensions to analyze CSV data.
- **Task:** Analyze employee salary data from a CSV file.
- **Instructions:**
 - Read employees.csv using the csv module
 - Convert rows into a list of dictionaries
 - Filter employees with salary > 50000 using list comprehension
 - Calculate and print average salary

```

1 import csv
2 def process_csv():
3     try:
4         file = open("emp.csv", "r")
5         reader = csv.DictReader(file)
6         employees = list(reader)
7         file.close()
8
9         high_salary = [
10             emp for emp in employees
11             if int(emp["salary"]) > 50000
12         ]
13
14         total = sum(int(emp["salary"]) for emp in employees)
15         average = total / len(employees)
16         print("Employees with salary > 50000")
17
18         for emp in high_salary:
19             print(emp)
20         print("\nAverage Salary:", average)
21
22     except FileNotFoundError:
23         print("CSV file not found")
24     except ValueError:
25         print("Invalid data in CSV file")
26 process_csv()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\admin\Desktop\cognizant python> c; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.e
e' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '50418' '--' 'c:\Users\admin\Desktop\cognizant python\c
v_data.py'
{'name': 'Pooja', 'salary': '100000'}
{'name': 'Swetha', 'salary': '85000'}
{'name': 'Vasan', 'salary': '75000'}
{'name': 'Selvi', 'salary': '65000'}

Average Salary: 81250.0

```

45. Expense Tracker

- **Objective:** Use file handling, date processing, lists, dictionaries, and comprehensions for expense analysis.
- **Task:** Analyze current month expenses and calculate totals per category.
- **Instructions:**

- Read expenses.csv (date, amount, category)
- Filter current month using datetime
- Group expenses by category using dictionary
- Calculate total amount per category
- Print summary

```

1 import csv
2 from datetime import datetime
3 def analyze_expenses():
4
5     try:
6         file = open("expenses.csv", "r")
7         reader = csv.DictReader(file)
8         category_totals = {}
9         current_month = datetime.now().month
10        for row in reader:
11
12            expense_date = datetime.strptime(row["date"], "%Y-%m-%d")
13            if expense_date.month == current_month:
14
15                category = row["category"]
16                amount = float(row["amount"])
17
18                if category in category_totals:
19                    category_totals[category] += amount
20                else:
21                    category_totals[category] = amount
22
23        file.close()
24        print("Expense Summary")
25
26        for category, total in category_totals.items():
27            print(f"{category}: {total}")
28
29    except FileNotFoundError:
30        print("Expenses file not found")
31
32    except ValueError:
33        print("Invalid data in file")
34
35 analyze_expenses()

```

```

PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.e
e' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '62572' '--' 'C:\Users\admin\Desktop\cognizant python\ve
penses_tracker.py'
Expense Summary
Food: 500.0
Travel: 1000.0
Medical: 300.0
Shopping: 700.0
Education: 400.0
PS C:\Users\admin\Desktop\cognizant python>

```

46. API Response Handler

- **Objective:** Use requests, JSON handling, classes, and error handling to process API responses.
- **Task:** Fetch weather data from an API and display temperature and conditions safely.
- **Instructions:**
 - Install & use requests
 - Fetch weather data (JSON)
 - Parse temperature and conditions
 - Handle 404 and network errors
 - Format output neatly

```

1  import requests
2  class WeatherAPI:
3      def __init__(self,city):
4          self.city=city
5      def get_weather(self):
6          url=f"https://wttr.in/{self.city}?format=json"
7          try:
8              response=requests.get(url,timeout=5)
9              if response.status_code==404:
10                 print("Error: City not found.")
11                 return
12             response.raise_for_status()
13             data=response.json()
14             temperature=data["current_condition"][0][["temp_c"]]
15             condition=data["current_condition"][0][["weatherDesc"]][0][["value"]]
16             print("Weather Report")
17             print("City:",self.city)
18             print("Temperature:",temperature,"°C")
19             print("Condition:",condition)
20         except requests.exceptions.ConnectionError:
21             print("Error: Network connection failed.")
22         except requests.exceptions.Timeout:
23             print("Error: Request timed out.")
24         except requests.exceptions.RequestException as e:
25             print("Error:",e)
26     weather=WeatherAPI("Tiruvannamalai")
27     weather.get_weather()

```

PS C:\Users\admin\Desktop\cognizant python> & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '58558' '--' 'c:\Users\admin\Desktop\cognizant python\api_handler.py'

Weather Report
 City: Tiruvannamalai
 Temperature: 34 °C
 Condition: Sunny

Additional real-time simulation hands-on questions

47. Complete Calculator Program

- **Objective:** Combine input, functions, error handling.
- **Task:** Build a robust calculator that supports + , - , * , /.
- **Instructions:**
 - Create function calculate(a, b, op) handling + , - , * , /.
 - Use input() for numbers/operation.
 - Handle ZeroDivisionError and ValueError.
 - Print formatted result.

The screenshot shows a VS Code editor with a project named 'cognizant python'. The Explorer sidebar on the left lists various Python files, including 'calculator.py'. The main editor window displays the code for 'calculator.py', which defines a 'calculate' function that handles addition, subtraction, multiplication, and division, with error handling for 'ZeroDivisionError' and 'ValueError'. Below the code editor, the TERMINAL panel shows the command prompt output where the program is executed with inputs 25, 41, and '+', resulting in 'Result: 66.0'.

```

1 def calculate(a, b, op):
2     try:
3         if op == "+":
4             return a + b
5         elif op == "-":
6             return a - b
7         elif op == "*":
8             return a * b
9
10        elif op == "/":
11            return a / b
12
13        else:
14            return "Invalid operator"
15
16    except ZeroDivisionError:
17        return "Cannot divide by zero"
18
19    try:
20        a = float(input("Enter first number: "))
21        b = float(input("Enter second number: "))
22        op = input("Enter operator (+, -, *, /): ")
23        result = calculate(a, b, op)
24        print("Result:", result)
25
26    except ValueError:
27        print("Please enter valid numbers")

```

```

PS C:\Users\admin\Desktop\cognizant python> c; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.
exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '49194' '-.' 'C:\Users\admin\Desktop\cognizant pytho
n\calculator.py'
Enter first number: 25
Enter second number: 41
Enter operator (+, -, *, /): +
Result: 66.0
PS C:\Users\admin\Desktop\cognizant python>

```

48. Shopping Cart System

- **Objective:** Use classes, lists, control flow, and the math module to build a simple shopping cart system.
- **Task:** Perform complete cart operations and print a receipt with GST.
- **Instructions:**
 - Create CartItem class with price and quantity
 - Create ShoppingCart class with:
 - add item
 - remove item
 - calculate total
 - Apply 18% GST
 - Print final receipt

```

1 class CartItem:
2     def __init__(self, name, price, quantity):
3         self.name = name
4         self.price = price
5         self.quantity = quantity
6
7 class ShoppingCart:
8     def __init__(self):
9         self.items = []
10    def add_item(self, item):
11        self.items.append(item)
12    def remove_item(self, name):
13        self.items = [
14            item for item in self.items
15            if item.name != name
16        ]
17    def calculate_total(self):
18        total = 0
19        for item in self.items:
20            total += item.price * item.quantity
21        return total
22
23    def print_receipt(self):
24        print("Shopping Cart Receipt")
25        for item in self.items:
26            print(f"{item.name} - {item.quantity} x {item.price}")
27        subtotal = self.calculate_total()
28        gst = subtotal * 0.18
29        final_total = subtotal + gst
30
31        print("\nSubtotal:", subtotal)
32        print("GST (18%):", gst)
33        print("Final Total:", final_total)
34
35 cart = ShoppingCart()
36 cart.add_item(CartItem("Laptop", 50000, 1))
37 cart.add_item(CartItem("Mouse", 500, 2))
38 cart.print_receipt()

```

```

PS C:\Users\admin\Desktop\cognizant python> c;; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '49743' '--' 'C:\Users\admin\Desktop\cognizant python\shopping.py'
Shopping Cart Receipt
Laptop - 1 x 50000
Mouse - 2 x 500

Subtotal: 51000
GST (18%): 9180.0
Final Total: 60180.0
PS C:\Users\admin\Desktop\cognizant python>

```

49. Temperature Converter GUI

- **Objective:** Use classes, user input, modules, and formatted output.
- **Task:** Convert temperatures between Celsius, Fahrenheit, and Kelvin.
- **Instructions:**
 - Converter class

- Convert $C \leftrightarrow F \leftrightarrow K$
- Use `input()` menu
- Format output to 2 decimals
- Use `tabulate` module for table display

The screenshot displays the Visual Studio Code editor with a Python project named 'cognizant python'. The Explorer sidebar on the left lists various Python files, including 'temperature_converter.py'. The main editor window shows the code for 'temperature_converter.py', which defines a 'Converter' class with methods for converting between Celsius, Fahrenheit, and Kelvin. The script prompts the user for a choice and a temperature, then performs the conversion and prints the result.

```
1 class Converter:
2     def c_to_f(self, c):
3         return (c * 9/5) + 32
4
5     def f_to_c(self, f):
6         return (f - 32) * 5/9
7
8     def c_to_k(self, c):
9         return c + 273.15
10
11     def k_to_c(self, k):
12         return k - 273.15
13
14 converter = Converter()
15 print("1. Celsius to Fahrenheit")
16 print("2. Fahrenheit to Celsius")
17 print("3. Celsius to Kelvin")
18 print("4. Kelvin to Celsius")
19
20 choice = int(input("Enter your choice: "))
21 temp = float(input("Enter temperature: "))
22
23 if choice == 1:
24     result = converter.c_to_f(temp)
25     print("Temperature:", round(result, 2), "F")
26
27 elif choice == 2:
28     result = converter.f_to_c(temp)
29     print("Temperature:", round(result, 2), "C")
30
31 elif choice == 3:
32     result = converter.c_to_k(temp)
33     print("Temperature:", round(result, 2), "K")
34
35 elif choice == 4:
36     result = converter.k_to_c(temp)
37     print("Temperature:", round(result, 2), "C")
38 else:
39     print("Invalid choice")
```

The terminal window at the bottom shows the execution of the script. It displays the prompts and the user's input, resulting in the output: 'Temperature: 96.8 F'.

```
PS C:\Users\admin\Desktop\cognizant python> & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python...
debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55067' '-.' 'C:\Users\admin\Desktop\cognizant python\temperature_converter.py'
2. Fahrenheit to Celsius
3. Celsius to Kelvin
4. Kelvin to Celsius
Enter your choice: 1
Enter temperature: 36
Temperature: 96.8 F
PS C:\Users\admin\Desktop\cognizant python>
```

50. Backup Utility

- **Objective:** Use file operations, modules, sets, and error handling to build a smart backup tool.
- **Task:** Copy files to a backup folder while skipping duplicates and logging operations.
- **Instructions:**
 - Use shutil module.
 - Copy files but skip duplicates using sets.
 - Handle FileNotFoundError, PermissionError.
 - Log operations to backup.log.


```

1 import shutil
2 copied_files = set()
3 def backup_file(source, destination):
4
5     try:
6         if source in copied_files:
7             print("Duplicate file skipped")
8             return
9
10        shutil.copy(source, destination)
11        copied_files.add(source)
12
13        log = open("backup.log", "a")
14        log.write(f"Copied: {source}\n")
15        log.close()
16        print("File copied successfully")
17
18    except FileNotFoundError:
19        print("File not found")
20
21    except PermissionError:
22        print("Permission denied")
23
24 backup_file("sample.txt", "backup_sample.txt")

```

```

PS C:\Users\admin\Desktop\cognizant python> c;; cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '54456' '-.' 'c:\Users\admin\Desktop\cognizant python\utility.py'
File copied successfully
PS C:\Users\admin\Desktop\cognizant python>

```

51. URL Shortener

- **Objective:** Use dictionaries, hashing, classes, and modules to build a simple URL shortening service.
- **Task:** Create a basic URL shortener that can shorten URLs and retrieve the original URL.
- **Instructions:**
 - Import hashlib
 - URLShortener class with dictionary storage
 - Generate 6-character hash from URL
 - Lookup (redirect simulation) functionality

```

1 import random
2 import string
3 url_database = {}
4 def generate_short_url():
5     characters = string.ascii_letters + string.digits
6     short_code = ''.join(random.choice(characters) for i in range(6))
7     return short_code
8
9 def shorten_url(long_url):
10    short_code = generate_short_url()
11    url_database[short_code] = long_url
12    return f"http://short.ly/{short_code}"
13
14 def retrieve_url(short_url):
15    short_code = short_url.split("/")[-1]
16    return url_database.get(short_code, "URL not found")
17
18 original_url = "https://www.google.com"
19 short_url = shorten_url(original_url)
20 print("Short URL:", short_url)
21
22 print("Original URL:", retrieve_url(short_url))

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '49743' '--' 'c:\Users\admin\Desktop\cognizant python\url.py'
Short URL: http://short.ly/www0eS
Original URL: https://www.google.com
PS C:\Users\admin\Desktop\cognizant python>

```

52. Gradebook System

- **Objective:** Use dictionaries, functions, control flow, and file I/O for student grade management.
- **Task:** Manage student grades, calculate GPA, save/load data, and compute class average.
- **Instructions:**
 - Dictionary: student → list of grades
 - Functions: add grade, calculate GPA, save/load
 - Handle invalid grade inputs
 - Print class average

```
1 class Student:
2     def __init__(self, name, marks):
3         self.name = name
4         self.marks = marks
5
6     def average(self):
7         return sum(self.marks) / len(self.marks)
8     def grade(self):
9         avg = self.average()
10        if avg >= 90:
11            return "A"
12        elif avg >= 75:
13            return "B"
14        elif avg >= 50:
15            return "C"
16        else:
17            return "Fail"
18    def display(self):
19        print("Student Name:", self.name)
20        print("Marks:", self.marks)
21        print("Average:", self.average())
22        print("Grade:", self.grade())
23
24 student1 = Student("Pooja", [85, 90, 88])
25 student1.display()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\admin\Desktop\cognizant_python> c::; cd 'c:\Users\admin\Desktop\cognizant_python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '51110' '--' 'c:\Users\admin\Desktop\cognizant_python\gradebook.py'

Student Name: Pooja
Marks: [85, 90, 88]
Average: 87.66666666666667
Grade: B

53. Task Scheduler

- **Objective:** Use classes, the datetime module, lists, and sorting to manage tasks.
- **Task:** Create a task scheduler that prioritizes tasks by due date and identifies overdue tasks.
- **Instructions:**
 - Task class with due_date and priority
 - Store tasks in a list
 - Sort tasks by due date using datetime
 - Filter overdue tasks
 - Print sorted task schedule

```

1 from datetime import datetime
2 class Task:
3     def __init__(self, name, due_date, priority):
4         self.name = name
5         self.due_date = datetime.strptime(due_date, "%Y-%m-%d")
6         self.priority = priority
7
8     def display(self):
9         print(f"Task: {self.name}")
10        print(f"Due Date: {self.due_date.date()}")
11        print(f"Priority: {self.priority}")
12        print()
13
14    tasks = []
15    tasks.append(Task("Python Assignment", "2026-06-10", "High"))
16    tasks.append(Task("Mini Project", "2026-06-05", "Medium"))
17    tasks.append(Task("Record Submission", "2026-06-01", "Low"))
18    tasks.sort(key=lambda x: x.due_date)
19    today = datetime.now()
20    print("Task Schedule\n")
21    for task in tasks:
22        task.display()
23        if task.due_date < today:
24            print("Status: Overdue\n")
25        else:
26            print("Status: Upcoming\n")

```

```

PS C:\Users\admin\Desktop\cognizant python> c:: cd 'c:\Users\admin\Desktop\cognizant python'; & 'c:\Users\admin\AppData\Local\Microsoft\WindowsApps\python3.1
exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53446' '--' 'c:\Users\admin\Desktop\cognizant py
n\task.py'
Task Schedule

Task: Record Submission
Due Date: 2026-06-01
Priority: Low

Status: Overdue

Task: Mini Project
Due Date: 2026-06-05
Priority: Medium

Status: Upcoming

Task: Python Assignment
Due Date: 2026-06-10
Priority: High

Status: Upcoming

```

54. Inventory Manager

- **Objective:** Use classes, inheritance, dictionaries, and sets to manage warehouse inventory.
- **Task:** Track products, stock levels, and generate low-stock alerts.
- **Instructions:**
 - Product base class
 - Perishable and Electronics inherit from Product
 - Track stock levels using a dictionary
 - Use sets for low-stock alerts
 - Print inventory summary

```

1 class Product:
2     def __init__(self, name, stock):
3         self.name = name
4         self.stock = stock
5 class Perishable(Product):
6     def category(self):
7         return "Perishable"
8 class Electronics(Product):
9     def category(self):
10        return "Electronics"
11 inventory = {}
12 low_stock = set()
13
14 p1 = Perishable("Milk", 5)
15 p2 = Electronics("Laptop", 20)
16 p3 = Perishable("Bread", 2)
17
18 inventory[p1.name] = p1.stock
19 inventory[p2.name] = p2.stock
20 inventory[p3.name] = p3.stock
21 for product, stock in inventory.items():
22     if stock < 5:
23         low_stock.add(product)
24 print("Inventory Summary\n")
25 for product, stock in inventory.items():
26     print(f"Product: {product}")
27     print(f"Stock: {stock}")
28     print()
29 print("Low Stock Alerts")
30 for item in low_stock:
31     print(item)

```

```

PS C:\Users\admin\Desktop\cognizant_python> python n\inventory.py
Inventory Summary

Product: Milk
Stock: 5

Product: Laptop
Stock: 20

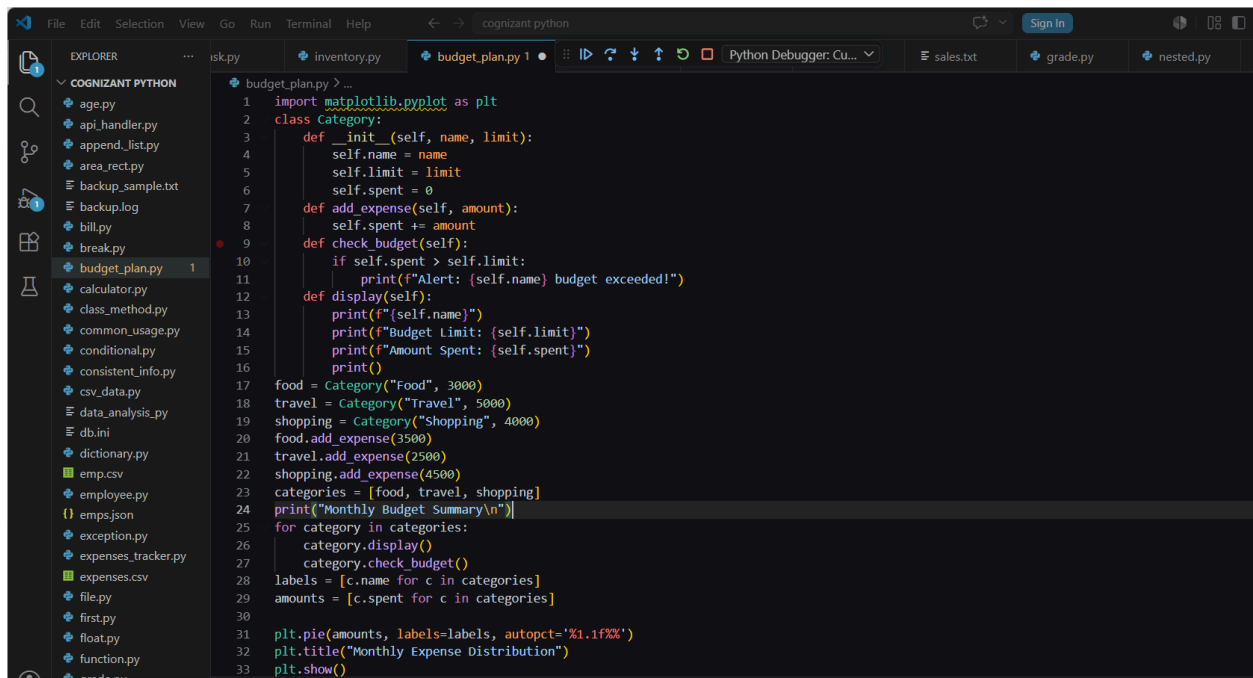
Product: Bread
Stock: 2

Low Stock Alerts
Bread

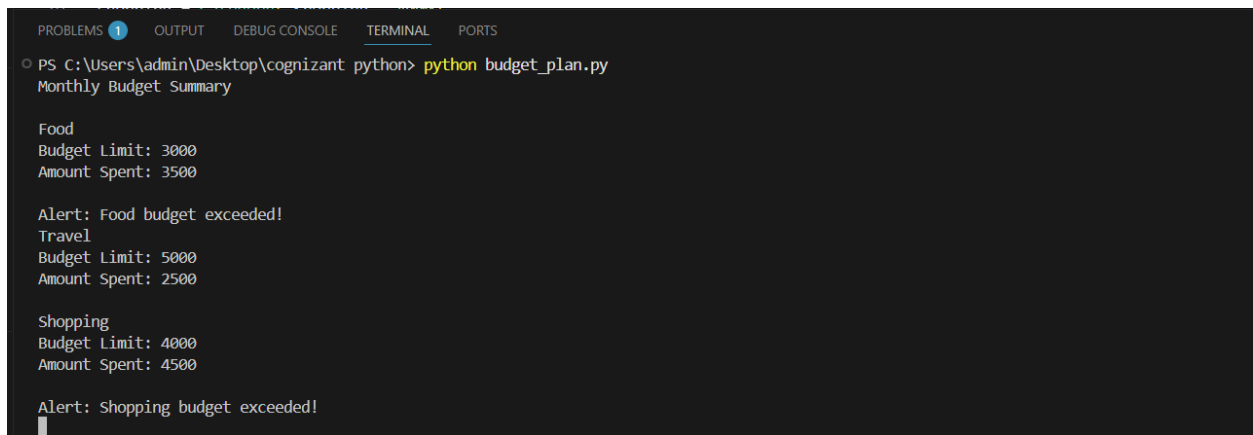
```

55. Budget Planner

- **Objective:** Use classes, control flow, external modules, and visualization to track a monthly budget.
- **Task:** Create a monthly budget planner that tracks spending, alerts overspending, and visualizes data.
- **Instructions:**
 - Category class with limit and spent
 - Input expenses using a loop
 - Use matplotlib for pie chart visualization
 - Alert when category exceeds budget



```
1 import matplotlib.pyplot as plt
2 class Category:
3     def __init__(self, name, limit):
4         self.name = name
5         self.limit = limit
6         self.spent = 0
7     def add_expense(self, amount):
8         self.spent += amount
9     def check_budget(self):
10        if self.spent > self.limit:
11            print(f"Alert: {self.name} budget exceeded!")
12    def display(self):
13        print(f"{self.name}")
14        print(f"Budget Limit: {self.limit}")
15        print(f"Amount Spent: {self.spent}")
16        print()
17 food = Category("Food", 3000)
18 travel = Category("Travel", 5000)
19 shopping = Category("Shopping", 4000)
20 food.add_expense(3500)
21 travel.add_expense(2500)
22 shopping.add_expense(4500)
23 categories = [food, travel, shopping]
24 print("Monthly Budget Summary\n")
25 for category in categories:
26     category.display()
27     category.check_budget()
28 labels = [c.name for c in categories]
29 amounts = [c.spent for c in categories]
30
31 plt.pie(amounts, labels=labels, autopct='%1.1f%%')
32 plt.title("Monthly Expense Distribution")
33 plt.show()
```



```
PS C:\Users\admin\Desktop\cognizant python> python budget_plan.py
Monthly Budget Summary

Food
Budget Limit: 3000
Amount Spent: 3500

Alert: Food budget exceeded!
Travel
Budget Limit: 5000
Amount Spent: 2500

Shopping
Budget Limit: 4000
Amount Spent: 4500

Alert: Shopping budget exceeded!
```

